Algorithm

Programmer Asia Mohammad

"وَقُلْ رَبِّ زِدْنِي عِلْماً"

Algorithm

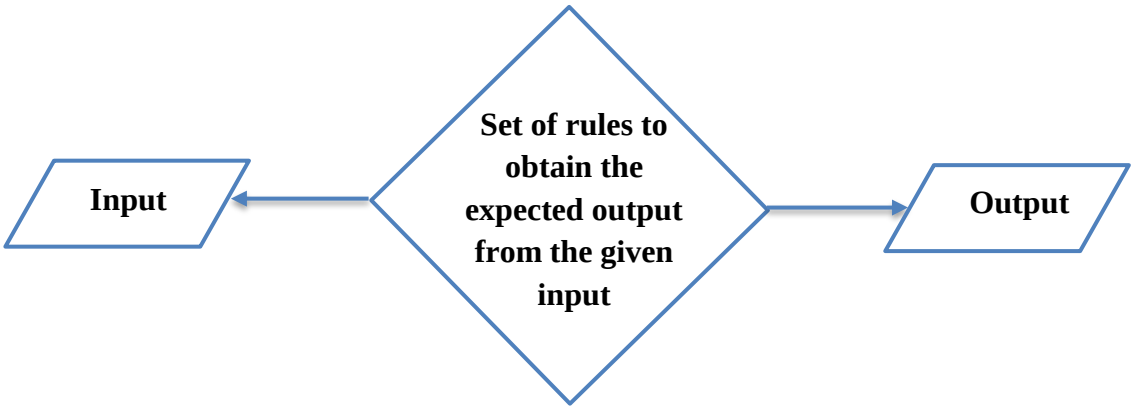
An algorithm is a set of steps of operations to solve a problem performing calculation , data processing , and automated reasoning tasks .

الخوارزمية هي مجموعة من خطوات العمليات لحل مشكلة ، وتشمل إجراء عمليات حسابية ومعالجة بيانات واستدلال آلي .

An algorithm is the best way to represent the solution of a particular problem in a very simple and efficient way .

وهي أفضل طريقة لتمثيل حل مشكلة معينة بطريقة بسيطة وفعالة لـ الغاية .

(مجموعة من الخطوات لحل مشكلة ، مثلاً بدي اعمل كيكة في خطوات معينة عشان اجهزها وهاي الخطوات بسميها خوارزمية ، رح تكون خطوات منظمة ومرتبة و واضحة عشان امشي عليها خطوة خطوة بالتنفيذ) .



(Input) المُدخلات **(Processing) المُعالجة** **(Output) المُخرجات**

An algorithm is said to be correct if , for every input instance , it halts with the correct output .

يُقال أن الخوارزمية صحيحة إذا توقفت عند الإخراج الصحيح لكل مَثيل إدخال .

في بعض الـ problem الها مُدخلات بدون مُخرجات ، اسمها Black hole problem مشاكل الثقب الأسود بتاخذ بس Input وما بتخرج الـ Output وفي problem الها مُخرجات بدون مُدخلات اسمها Zero-input problem مشاكل بتخرج Output بدون ما تأخذ Input .

Algorithms must be :

Input : An algorithm can accept zero or more external inputs .

المُدخلات : يمكن لـ الخوارزمية أن تقبل صفر أو أكثر من المدخلات الخارجية .

Output : An algorithm must produce at least one output , this is the result or solution generated after processing the input(s) .

المُخرجات : يجب أن تنتج الخوارزمية ناتجاً واحداً على الأقل ، هذا هو الحل أو النتيجة الناتجة بعد معالجة المُدخلات .

Definiteness : Each instruction is clear and unambiguous .

الوضوح : كل تعليمات الخوارزمية يجب أن تكون واضحة وغير غامضة .

Finiteness : The algorithm must terminate after a finite number of steps for all inputs , it cannot run indefinitely or be stuck in a loop ; it must always reach a conclusion after a certain amount of steps .

النهاية : يجب أن تنتهي الخوارزمية بعد عدد محدود من الخطوات لجميع المُدخلات ، ولا يمكن تشغيلها إلى ما لا نهاية أو أن تظل عالقة في حلقة ؛ بل يجب أن تصل دائماً إلى نتيجة بعد عدد معين من الخطوات .

Efficiency : Every instruction should be simple , quick , and yield results more efficiently than human computations (Time , Cost , Space) .

الكفاءة : يجب أن تكون كل تعليمات بسيطة وسريعة وتعطي نتائج أكثر كفاءة من الحسابات البشرية (الوقت والتكلفة والمساحة) .

الخوارزمية من شو لازم تتكون ؟

المُدخلات وهاي الأساس عشان يكون عندي مُخرجات ، وبعملية المُعالجة مش دايماً أي خوارزمية بكتبها رح تكون حل ، في شروط منها عدد الأوامر او الخطوات تكون محدودة يعني مثلاً لو بدي اكتب كود حل لمسألة يكون كود لا نهائي ؟ ما وصلت لحل لـ المسألة .

ولازم كمان الخطوات نفسها يلي بكتبها بالكود تكون واضحة ومفهومة ومرتبة وصحيحة ومصطلحات أي حدا بعدي يقرأها يفهم شو مكتوب (الخوارزمية يلي بكتبها مش شرط لـ اشي برمجي وكود عادي بكتبها لـ أي مشكلة بنفس الشروط) .

وأهم شغلة بالخوارزمية انه تكون الها كفاءة وتعطي نتائج سريعة وما تاخذ وقت ومساحة وتكلفة مثلاً اذا بدي اروح من جنين لـ طولكرم وارجع لنفس المكان عجنين فبختار اقصر طريق عشان الوقت لهيك رح اختار أفضل solution استخدمها عشان أوصل لـ حل مناسب وصحيح بسرعة وكل problem عندي ممكن يكون الها اكثر من solution .

اول اشي بيكون عندي مشكلة واقعية

Real World Problem

Understand the Problem فهم المشكلة

(شو المُدخلات والمُخرجات والشروط ؟)

Select appropriate Design Techniques and Data Structures

بختار الطريقة وبُنية البيانات .

Write the Computational Solutions (Algorithms)

بكتب خطوات الحل على شكل خوارزمية
(Flowchart أو Pseudo Code)

Validate the Algorithms

بتأكد إنها بتعطي الجواب صح بكل الحالات

Analyze the Algorithms

بحلل الخوارزمية Time Complexity
و Space Complexity .

Select the efficient Algorithm

بختار أنسب حل لـ الأداء .

Implement the Algorithm

بكتب الخوارزمية بلغة برمجة

Testing Program

Data Structures يلي ممكن استخدمها في الحل

Array المصفوفة بتخزن عناصر متتالية بنفس النوع

Linked List عناصرها مربوطة ببعض بروابط **Pointers**

Stack مبدأه يلي بدخل آخر بطلع أول **LIFO**

Queue مبدأه يلي بدخل أول بطلع أول**FIFO**

Binary Tree كل عقدة إلها تفرعين كحد أقصى .

Binary Search Tree بتشبهه الشجرة الثنائية بس منظمة ومرتبة حسب القيم .

Heap شجرة بتنظم القيم حسب أولوية معينة **Min/Max**

Red Black Tree (شجرة أحمر-أسود) نوع خاص من الأشجار المتوازنة .

Quad Tree شجرة رباعية بتقسم المساحة لأربع مناطق مستخدمة بالرسومات .

Graph الرسم البياني

Un-directed Graph الحواف بدون اتجاه .

Directed Graph الحواف إلها اتجاه .

Planar Graph (مستوي) ممكن ارسمه بدون ما تتقاطع الحواف .

طرق تصميم الخوارزميات

Iterative Method بكرر خطوات باستخدام **loops**

Recursive Method الدالة بتستدعي نفسها لحل أجزاء أصغر .

Brute-force Method بجرب كل الاحتمالات لـ اوصل لـ الحل .

Divide and Conquer بقسم المشكلة لأجزاء أصغر واحلها .

Greedy Method بختار الحل الأفضل محلياً بكل خطوة خوارزمية المسار الأقصر .

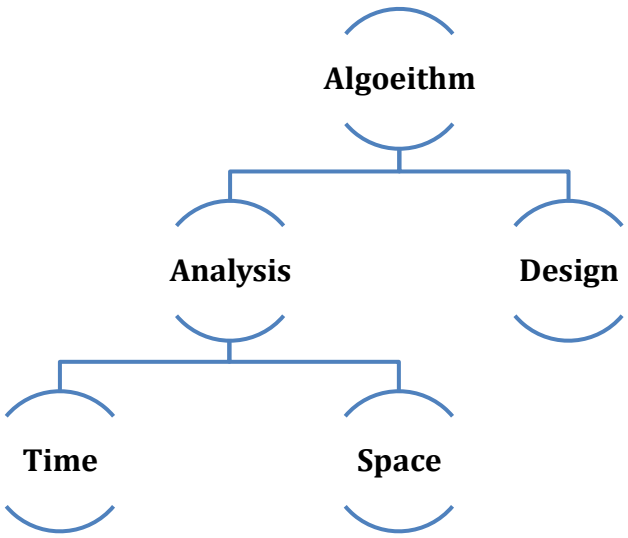
Dynamic Programming بحل المشكلة بتخزين نتائج المشاكل الجزئية عشان استخدمها بعدين

Branch and Bound Method بستخدمه لـ المشاكل الكبيرة يلي إلها حلول محتملة كتيرة ، بنقصي الحالات غير المفيدة .

Backtracking Method بجرب حل ، وإذا ما زبط برجع خطوة لورا

Optimization Method ببحث عن أفضل حل أقل تكلفة أعلى ربح .

Random Method بستخدم التوزيع العشوائي لـ اوجد حلول تقريبية .



- **Analysis : الخوارزمية موجودة وانا مطلوب مني أحلها**
- **Design : أنا رح أبني الخوارزمية من البداية**

Analysis : predict the cost of an algorithm in terms of resources and performance .

Design: creating an efficient algorithm to solve a problem in an efficient way using minimum time and space .

التحليل : التنبؤ بتكلفة الخوارزمية من حيث الموارد والأداء .

التصميم : إنشاء خوارزمية فعّالة لحل المشكلة بكفاءة عالية باستخدام الحد الأدنى من الوقت والمساحة .

Time Complexity is a function describing the amount of time required to run an algorithm in terms of the size of the input .

تعقيد الزمن (Time Complexity) دالة تصف كمية الوقت المطلوبة لتشغيل خوارزمية ، بناءً على حجم المُدخلات .

What make algorithm fast ?

Space Complexity is a function describing the amount of memory an algorithm takes in terms of the size of input to the algorithm .

تعقيد المساحة (Space Complexity) دالة تصف كمية الذاكرة التي تستهلكها الخوارزمية ، بناءً على حجم المُدخلات .

How much memory is used ?

How to test a program ?

كيف نختبر برنامج ؟

After implementation (posterior analysis) , Testing a program consists of two phases :

بعد تنفيذ البرنامج (تحليل لاحق) ، اختبار البرنامج بتكوّن من مرحلتين :

- **Debugging (التصحيح)**
- **Profiling (التحليل أو قياس الأداء)**

Debugging is the process of executing programs on sample data sets to determine whether the faulty results occur if so to correct them .

التصحيح هو عملية تنفيذ البرامج على مجموعات بيانات عينة لتحديد ما إذا كانت النتائج الخاطئة تحدث ، وإذا كان الأمر كذلك لـ تصحيحها .

Profiling or performance measurement is the process of executing a correct program on data sets and measuring time and space it takes to compute the results .

قياس الأداء أو تحديد الملف الشخصي فهو عملية تنفيذ برنامج صحيح على مجموعات بيانات وقياس الوقت والمساحة اللازمة لحساب النتائج .

لو مثلاً عندي خوارزمية مُعينة لحساب مجموع عددين x و y وعرفتهم int x , int y وكتبت خطوات الحل وطبعت المجموع ، الحل صح بس بشرط الاعداد تكون int دايماً .

لو دخلت double فالحل رح يكون مش صحيح لكل الاعداد ، انا بأول خطوة بشوف الحل وبدخل مرحلة التصحيح والتعديل عالكود الموجود .

في طريقة سهلة بالـ intellij عشان أفحص الكود ، بحدد بالبداية Break Point هاي نقطة توقف عند سطر معين عشان اراقب شو رح يصير وبعدها بشغل البرنامج باستخدام Debug مش باستخدام Run

رح يفتح نافذه فيها الـ variables ، call stack ، debugger control

وهيك بطريقة اسهل بشوف وين الـ Error بالكود أسهل من الفحص النظري لـ الكود عشان اشوف مشاكل الخوارزمية يلي انا كتبتها مثلاً .

There are three way to specify an algorithm

- **Natural language**

It is very simple and easy to specify an algorithm using natural language But many times , specification of algorithm by using natural language is not clear and thereby we get brief specification .

Such a specification creates difficulty while implementing it , Hence many programmers prefer specification of algorithm by means of Pseudocode .

من السهل والبسيط جداً تحديد خوارزمية باستخدام اللغة الطبيعية ولكن في كثير من الأحيان ، يكون تحديد الخوارزمية باستخدام اللغة الطبيعية غير واضح ، وبالتالي نحصل على مواصفات غير دقيقة .

هذا النوع من التحديد يسبب صعوبة أثناء تنفيذ الخوارزمية ولذلك ، يفضل العديد من المبرمجين تحديد الخوارزمية باستخدام الكود الكاذب (Pseudocode) .

الطريقة الأولى Natural language بلغتي عادي بكتب فيها بشرح الخطوات يلي رح اعملها (ادخل العددين ، اجمع العددين ، اطبع العددين ...) ، سهلة لخوارزمية بسيطة بس كل ما كانت مُعقدة ما رح اعرف اكتب كثير جمل خاصة يلي فيها تكرار تنفيذ لشرط مُعين loop .

Example 1 :

We need to solve a computational problem

" Convert a weight in pounds to Kg "

An algorithm specifies how to solve it :

- **Step 1 : Read weight-in-pounds**
- **Step 2 : Calculate weight-in-Kg = weight-in-pounds * 0.455**
- **Step 3 : Print weight-in-Kg**

A computer program is a computer-executable description of an algorithm .

برنامج الحاسوب هو وصف قابل لتنفيذ الخوارزمية .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Example 2 :

We need to solve a algorithm selection sort

Input : A list L of integers of length n

Output : A sorted list L1 containing those integers present in L

(ترتيب array)

- **Step 1 : Find the minimum value in the list L**
- **Step 2 : Swap it with the value in the current position**
- **Step 3 : Repeat this process for all the elements until the entire list is sorted**
- **Step 4 : Return the sorted list L1**
- **Step 5 : Stop**

❖ Flowchart

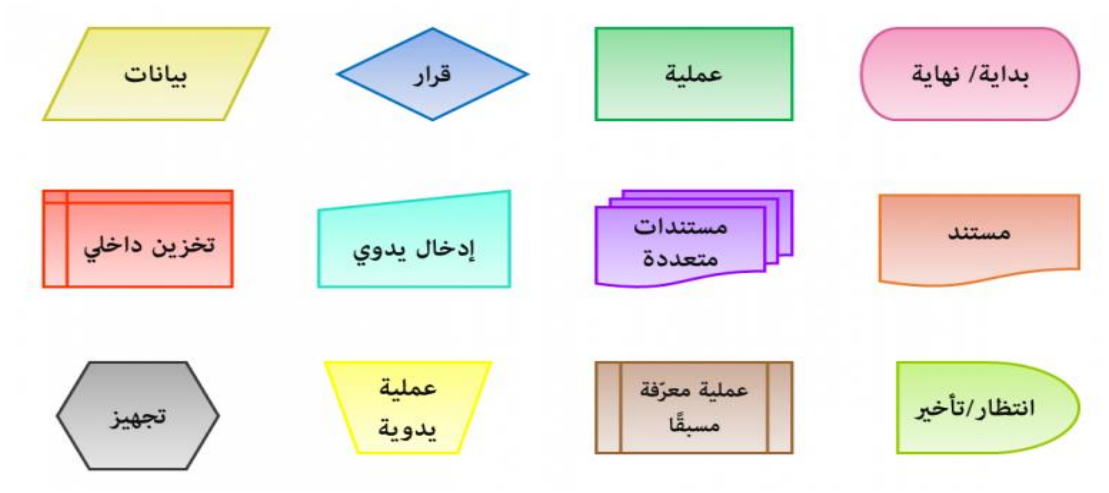
In the earlier days of computing , the dominant method for specifying algorithms was a flowchart , This representation technique has proved to be inconvenient .

Flowchart is a graphical representation of an algorithm , It is a method of expressing an algorithm by a collection of connected geometric shapes containing descriptions of the algorithm's steps .

في الأيام الأولى من الحوسبة ، كانت الطريقة السائدة لتحديد الخوارزميات هي مخطط الانسياب لكن هذه الطريقة أثبتت أنها غير مريحة (أو غير ملائمة) .

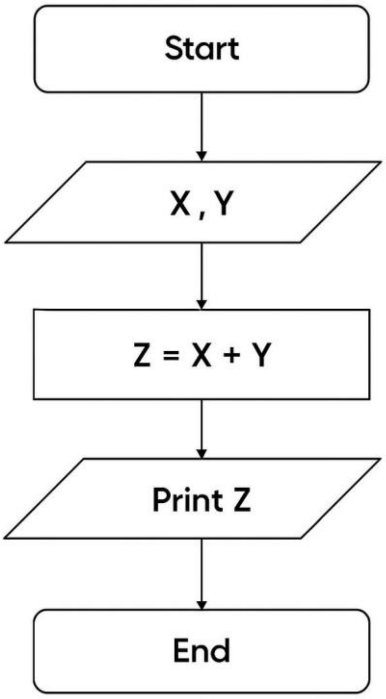
مخطط الانسياب هو تمثيل رسومي لخوارزمية وهو طريقة للتعبير عن الخوارزمية من خلال مجموعة من الأشكال الهندسية المتصلة التي تحتوي على أوصاف لخطوات الخوارزمية .

الطريقة الثانية Flowchart سهلة وممتعة بس لخوارزمية قصيرة وبسيطة ، صعب اعمل Flowchart لخوارزمية طويلة رح يكون مخطط كبير وطويل غلبة .



Example 1 :

We need to solve an algorithm to add two numbers



"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

❖ **Pseudocode**

Is an informal and human readable description of an algorithm leaving many details of it , writing a pseudocode has no restriction of styles and its only objective is to describe the high level steps of algorithm .

هو وصف غير رسمي وسهل القراءة من قبل الإنسان لخوارزمية يترك الكثير من التفاصيل عنها ، وكتابة الكود الزائف ليس له قيود على الأنماط وهدفه الوحيد هو وصف الخطوات عالية المستوى للخوارزمية .

الطريقة الثالثة Pseudocode وصف لـ الخوارزمية بطريقة بتشبهه الكود ، بسميه الكود الزائف وفيه شروط مُعية لـ الكتابة

1. **An algorithm is a procedure it has two parts; the first part is head and the second part is body .**

   الخوارزمية إجراءٌ يتكون من جزأين ؛ الجزء الأول هو الرأس والجزء الثاني هو الجسم .

   - **الرأس (Head) : فيه اسم الخوارزمية وقائمة المتغيرات أو المدخلات**
   - **الجسم (Body) : فيه الخطوات الفعلية يلي بتحل الخوارزمية**

2. **The Head section consists of keyword Algorithm and Name of the algorithm with parameter list**
   The head section also has the following
   // Problem Description :
   // Input :
   // Output :

يتكون القسم الرئيسي من الكلمة المفتاحية "الخوارزمية " واسم الخوارزمية مع قائمة المعاملات ، يعني بكتب أول اشي كلمة algorithm بعدها اسم المشكلة وبفتح اقواس تستقبل القيم ( )

{

بحط 3 comment

// وصف المشكلة : شو يلي الخوارزمية بتحاول تحله

// الإدخال : البيانات يلي بدها تدخل لـ الخوارزمية

// الإخراج : النتيجة يلي رح ترجعها الخوارزمية

بعدها بكتب خطوات الحل

}

3. **In the body of an algorithm various programming constructs like if, for, while and some statements like assignments are used .**

في جسم الخوارزمية يتم استخدام العديد من هياكل البرمجة مثل if و for و while وبعض العبارات مثل التعيينات .

- **if : للتفرعات الشرطية**
- **for , while : للحلقات التكرارية**

4. **The compound statements may be enclosed with { and } brackets If , for , while can be open and closed by { , } respectively , Proper indention is must for block .**

يمكن وضع العبارات المركبة بين قوسين { و } إذا كان من الممكن فتح وإغلاق { و } على التوالي ، فيجب أن تكون المسافة البادئة الصحيحة للكتلة .

5. **Comments are written using // at the beginning .**

تتم كتابة التعليقات باستخدام // في البداية .

6. **The identifier should begin by a letter and not by digit , It contains alpha numeric letters after first letter , No need to mention data types .**

يجب أن يبدأ المعرف بحرف وليس برقم ، ويحتوي على أحرف أبجدية رقمية بعد الحرف الأول ، ولا حاجة لذكر أنواع البيانات .

7. **The left arrow "←" used as assignment operator .**

السهم الأيسر " ← " يستخدم كعامل تعيين .

- **يستخدم ← لـ اسناد القيمة بدل =**

x ← 5 (x تساوي 5)

x = 5 (هل x تساوي 5 ؟)

8. **Boolean operators (True , False) , Logical operators (AND , OR , NOT) and Relational operators (< , <= , > , >= , = , ≠ , < >) are also used .**

يتم أيضاً استخدام العوامل المنطقية (True ، False) ، والعوامل المنطقية (AND ، OR ، NOT) ، والعوامل العلائقية (> ، => ، < ، =< ، = ، ≠ ، < >) .

بالكود :	**!**	**^**	**||**	**&&**
Pseudocode :	**NOT**	**XOR**	**OR**	**AND**

9. **Input and Output can be done using read and write .**

يمكن إجراء الإدخال والإخراج باستخدام القراءة والكتابة .

10. **Array [ ] , if then else condition , branch and loop can be also used in algorithm .**

يمكن أيضاً استخدام Array [ ] و if then else condition و branch و loop في الخوارزمية .

The general form of the for loop is :

```
for variable ← value1 to value2 step value3 do
{
   Statement  1 ;
   Statement  2 ;
   ...
   Statement  n ;
}
```

```
for i ← 1 to 10 do
   write(i);   // displaying numbers from 1 to 10
i ← i + 1 ;
```

```
for i ← 1 to 10 step 2 do
{
   write(i);   // displaying numbers from 1 to 10
   i ← i + 2;
}
```

The general form of the for while loop is :

```
while (condition) do
{
   statement 1;
   statement 2;
   ...
   statement n;
}
```

```
i ← 1;
while (i <= n) do
{
   write(i);
   i ← i + 1;
}
```

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Example 1 :

Algorithm for calculating the sum of two numbers (خوارزمية لـ جمع عددين)

```
Algorithm Sum ( a , b )
{
    //  Problem Description : This algorithm performs addition of two numbers
    //  Input : Two integers a and b
    // Output : Addition of two integers
    c ← a + b ;
    return c ;
}
```

Example 2 :

Algorithm for find greatest common divisor (خوارزمية لـ القاسم المشترك الأكبر)

```
Algorithm gcd ( a , b )
{
    // Problem Description : Calculate the greatest common divisor
    // Input : Two integers a and b
    // Output : The greatest common divisor

    while b ≠ 0 do

    r ← a mod b

    a ← b          // b تساوي a

    b ← r          // تساوي باقي القسمة b

    end while

    return a ;

}
```

Example 3 :

Algorithm for find the sum of all even numbers from 1 to n

(خوارزمية إيجاد مجموع الاعداد الزوجية من 1 الى n)

```
Algorithm SumEvenNumbers ( n )
{
      // Problem Description : Find the sum of all even numbers from 1 to n
      // Input : n
      // Output : sum of even numbers

      sum ← 0

      i ← 1

     Using while loop

     while i ≤ n do

              if i mod 2 = 0 then

              sum ← sum + i

              end if

             i ← i + 1

    end while

    write sum  or print sum

}
```

Example 4 :

Algorithm for find the sum of all odd numbers from 1 to n

(خوارزمية إيجاد مجموع الاعداد الفردية من 1 الى n)

الاختلاف بس

if i mod 2 ≠ 0 then // إذا العدد فردي

Example 5 :

Algorithm for print Hello Asia

```
Algorithm PrintMessage
{
        // Problem Description: Print a message once
       // Input: None
      // Output: The message
write Hello Asia
}
```

Example 6 :

Algorithm for print number

(5 , 10 , 15 , 20 , 25 , 30 , 35 , 40 , 45 , 50 خوارزمية طباعة الاعداد)

```
Algorithm PrintNumbersStep5
{
      // Problem Description : Print numbers from 5 to 50 with step of 5
     // Input : None
    // Output : List of numbers
   for i ← 5 to 50 step 5 do
       write i
  end for
}
```

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Analyzing of Algorithms

إحنا بنعمل تحليل لـ الخوارزميات عشان نقارن بين أكثر من خوارزمية مرشحة لحل مشكلة مُعينة ، وهيك بنقدر نختار الأكثر كفاءة بينهم .

- وقت التنفيذ (Computing time) وهاي بنسميها تعقيد زمني (Time Complexity)
- مساحة التخزين (Storage space) وهاي بنسميها تعقيد المساحة (Space Complexity)

لو مثلاً بدي اروح من جنين لـ طولكرم بشوف اقصر طريق وكم بتاخذ وقت او اذا فيها حواجز او لا ، عشان احدد احسن طريق اروح منها بوقت اقل ومشاكل اقل وافر جهد ووقت وتعب .

Efficiency of Algorithm

- **The efficiency of an algorithm is a measure of the amount of resources consumed in solving a problem of size n**
- **An algorithm must be analyzed to determine its resource usage**
- **Two major computational resources are execution time and memory space**

كفاءة الخوارزمية

- **كفاءة الخوارزمية هي مقياس لكمية الموارد المستهلكة لحل مشكلة حجمها n**
- **يجب تحليل الخوارزمية لتحديد استخدامها للموارد**
- **من أهم موارد الحوسبة وقت التنفيذ ومساحة الذاكرة**

لما اكتب خوارزمية مش المهم بس إنها تشتغل وتطلع النتيجة صح المهم كمان تكون سريعة وما تستهلك ذاكرة كثير ، كل ما كانت الخوارزمية أسرع وتستهلك ذاكرة أقل كل ما كانت أكفأ .

كفاءة الخوارزمية بتعتمد بشكل كبير على حجم البيانات المدخلة n ونوعها (صور ، اعداد ، نصوص ..) لانه رح تفرق بالإدخال ، كل ما كبر حجم بيانات الخوارزمية رح تحتاج وقت أطول عشان تشتغل وكمان ممكن تحتاج ذاكرة أكبر عشان تخزن البيانات أو المعالجات الوسيطة .

لو بدي اعتبر عندي طلاب وقدمو امتحان وعددهم 15 رح اخلّص تصليح الامتحان بوقت اقل من تصليح أوراق 25 طالب (بيعتمد على حجم البيانات المدخلة) .

عشان هيك ، لما نعمل تحليل كفاءة بنحاول نشوف كيف وقت التنفيذ ومساحة الذاكرة بيتغيروا لما n يزيد ، وهاد يلي بنعبر عنه بصيغة Big - O .

Time Complexity

- **Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input .**
- **Running time of an algorithm depends upon**
  - **Input Size**
  - **Nature of Input**

- **Generally , time grows with the size of input, for example , sorting 100 numbers will take less time than sorting of 10,000 numbers .**
- **So , running time of an algorithm is usually measured as a function of input size .**
- **Instead of measuring actual time required in executing each statement in the code , we consider how many times each statement is executed .**
- **So, in theoretical computation of time complexity , running time is measured in terms of number of steps / primitive operations performed .**

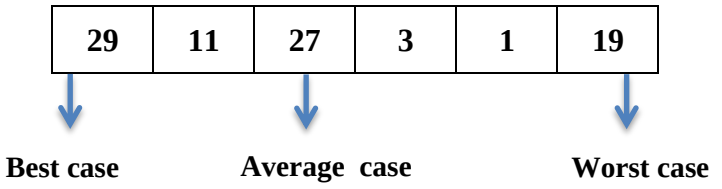
- **تعقيد الزمن للخوارزمية يقيس كمية الوقت التي تحتاجها الخوارزمية للتنفيذ كدالة لحجم المدخلات .**
- **زمن تنفيذ الخوارزمية يعتمد على :**
  - **حجم المدخلات**
  - **طبيعة المدخلات**

- **بشكل عام ، الزمن يزداد مع زيادة حجم المدخلات فمثلاً ترتيب 100 (sorting) رقم سيأخذ وقت أقل من ترتيب 10000 رقم .**
- **لذلك زمن تشغيل الخوارزمية غالباً يتم قياسه كدالة لحجم المدخلات .**
- **بدلاً من قياس الوقت الفعلي المطلوب لتنفيذ كل تعليمة في الكود ، نقيس كم مرة يتم تنفيذ كل تعليمة .**
- **لذلك في الحسابات النظرية لتعقيد الزمن ، يتم قياس زمن التشغيل بعدد الخطوات أو العمليات الأساسية التي تم تنفيذها .**

Time Complexity

عدد الأوامر يلي رح ينفذها البرنامج بزمن مُعين ويعطي نتيجة .

عندي array وبدي ابحث عن عنصر داخلها وخلال عملية البحث ممكن يكون العنصر الأول هو نفسه العنصر يلي ببحث عنه او الأخير هو العنصر يلي ببحث عنه او ممكن يكون موجود بالنّص .



Big O notation هو

Big O notation : ابعد خيار ممكن اوصله بعملية البحث عن العنصر worst case داخل ال array او خلال اي حل لوصف الزمن الذي تستغرقه الخوارزمية .

Analyzing algorithms : 3 cases

- **Worst case efficiency**

It is the maximum number of steps that an algorithm can take for any collection of data values .

- **Best case efficiency**

It is the minimum number of steps that an algorithm can take any collection of data values .

- **Average case efficiency**

The efficiency averaged on all possible inputs .

- **كفاءة أسوأ الحالات**

هي أقصى عدد من الخطوات التي يمكن أن تتخذها الخوارزمية لأي مجموعة من قيم البيانات .

- **كفاءة أفضل الحالات**

هي أقل عدد من الخطوات التي يمكن أن تتخذها الخوارزمية لأي مجموعة من قيم البيانات .

- **كفاءة متوسط الحالات**

هي متوسط الكفاءة لجميع المدخلات الممكنة .

Best case → Omega Notation Ω
Average case → Theta Notation Θ
Worst case → Big O Notation O

Best Case	Average Case	Worst Case
Resource usage is minimum	Resource usage is average	Resource usage is maximum
Algorithm's behavior under optimal condition	Algorithm's behavior under random condition	Algorithm's behavior under the worst condition
Minimum number of steps or operations	Average number of steps or operations	Maximum number of steps or operations
Lower bound on running time	Average bound on running time	Upper bound on running time
Generally do not occur in real	Average and worst-case performances are the most used in algorithm analysis.	

Problem	Best Case	Average Case	Worst Case
Linear Search	Element at the first position	Element in any of the middle positions	Element at last position or not present
Sorting	Already sorted	Randomly arranged	Sorted in reverse order

Space Complexity

- **Space complexity refers to the amount of space taken by an algorithm and produce a result .**
- **Relation between input size and used space to solve the problem .**
- **An improvement in space complexity reduces the amount of space (memory , disk space , etc .) taken by an algorithm .**

- يشير تعقيد المساحة إلى مقدار المساحة التي تشغلها الخوارزمية لإنتاج نتيجة .
- العلاقة بين حجم الإدخال والمساحة المستخدمة لحل المشكلة .
- يؤدي تحسين تعقيد المساحة إلى تقليل مقدار المساحة (الذاكرة ، مساحة القرص ، إلخ) التي تشغلها الخوارزمية .

Space Complexity

كمية المساحة (الذاكرة) يلي رح يستخدمها البرنامج عشان ينفذ الأوامر ويعطي نتيجة .

Linear Search Algorithm

- ببلش من أول عنصر في المصفوفة
- بقارن القيمة المطلوبة (Target) مع العنصر الحالي
- إذا كانت مساوية برجع الموقع (Index)
- إذا مش مساوية بكمّل للعنصر يلي بعده
- إذا وصلت لآخر المصفوفة وما لقيت القيمة يعني القيمة مش موجودة

Input : Array of n elements , Target value

Output : Index of Target if found , otherwise -1

```
Algorithm LinearSearch ( A , X )
{
for i ← 0 to last index of A do
   If Array[i]  ←  elements X then
      return i   // العنصر موجود ورجع موقعه
   end if
end for

return -1   // العنصر مش موجود
}
```

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Code

```
public class Main {
    public static void main(String[] args) {
        int[] arr = new int[100000000];

        for (int i = 0 ; i < arr.length ; i++) {
            arr[i] = i ;
        }
        System.out.println( Arrays.toString(arr) ) ;

        int x = 99999999 ; // العنصر يلي رح ادور عليه

        double t1 = System.currentTimeMillis( ) ; // for الوقت الحالي قبل ما ادخل

        for (int i = 0 ; i < arr.length ; i++) {
            if (arr[i] == x) {
                System.out.println( "Element " + x + " found in index : " + i ) ;
                break ;
            }
        }

        double t2 = System.currentTimeMillis( ) ; // for وقت النهاية بعد ما خلصت
// t2 – t1 =  الوقت المُستهلك
        System.out.println("The time is : " + (t2 - t1) + " ms");
    }
}
```

```
swap( int a , int b ) {

    int temp = a ;

    a = b ;

    b = temp ;
}
```

C1 رح يتنفذ مرة وحدة بس time (1)

C2 رح يتنفذ مرة وحدة بس time (1)

C3 رح يتنفذ مرة وحدة بس time (1)

Time

Cost

لو بدي احسب Running Time القيمة الفعلية لكل خطوة

Running Time	=	Time	*	Cost		
		1	*	C_1	=	C_1
		1	*	C_2	=	C_2
		1	*	C_3	=	C_3

Running time $= C_1 + C_2 + C_3 = C_4$

The time function : f (n) = 3

```
for ( Initialization ; Condition ; Increment / Decrement ) {

    // Body

}
```

رح يتنفذ كل مره بس يتحقق الشرط

رح يتنفذ مرة وحدة بس تعريف لـ المتغير .

رح يتنفذ كل مره بس يتحقق الشرط

رح يتنفذ كل مره بس يتحقق الشرط + مرة لما يفشل بس اتحقق منه باعتبار دخلت عالشرط ونفذته وفشل

```
int s = 0 ;
```

time (1) رح يتنفذ مرة وحدة بس C1

```
for ( int i=0 ; i<n ; i++ ) {

    s = s + A[i] ;

}
```

```
return s ;
```

time (1) رح يتنفذ مرة وحدة بس C6

لنفرض n = 3

	Condition	Body	Increment
	i < n	s = s + a[i]	i ++
i = 0	✓	✓	0 + 1 = 1 ✓
i = 1	✓	✓	1 + 1 = 2 ✓
i = 2	✓	✓	2 + 1 = 3 ✓
i = 3	✓ بدخل لـ الشرط عشان التحقق بس بفشل	ما بكمل لانه الشرط ما كان صح	ما بكمل لانه الشرط ما كان صح
عدد مرات التنفيذ	4	3	3
n = 3	n + 1	n	n

time (1) رح يتنفذ مرة وحدة بس i = 0 C2

time (n + 1) رح يتنفذ i < n C3

time (n) رح يتنفذ s = s + a[i] C4

time (n) رح يتنفذ i ++ C5

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	1	*	C_2	=	C_2
	$(n + 1)$	*	C_3	=	$(n + 1) C_3 = nC_3 + C_3$
	n	*	C_4	=	$n C_4$
	n	*	C_5	=	nC_5
	1	*	C_6	=	C_6

$$\text{Running Time} = C_1 + C_2 + nC_3 + C_3 + nC_4 + nC_5 + C_6$$

$$= C_1 + C_2 + nC_3 + C_3 + nC_4 + nC_5 + C_6$$

الثوابت بخليهم بثابت جديد C_7

$$= C_7 + nC_3 + nC_4 + nC_5$$

كأنه بخرج عامل مشترك الـ n

$$= C_7 + n (C_3 + C_4 + C_5)$$

الثوابت بخليهم بثابت جديد C_8

$$\text{Running time} = C_7 + nC_8$$

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

```
int s = 0 ;
```

time (1) رح يتنفذ مرة وحدة بس C1

```
for ( int i=0 ; i<n ; i++ ) {
```

time (n) رح يتنفذ C4

time (1) رح يتنفذ مرة وحدة بس C2

time (n+ 1) رح يتنفذ C3

Body * n

حكينا الـ Body رح يتنفذ بمقدار time (n)

```
    for ( int j=0 ; j<n ; j++ ) {

   s = s + a[i][j] ;
 }
}
return s ;
```

time (1) رح يتنفذ مرة وحدة بس C9

C5 j = 0 رح يتنفذ مرة وحدة بس time (1)

C6 j < n رح يتنفذ time (n + 1)

C7 s = s + a[i][j] رح يتنفذ (n)

C8 j ++ رح يتنفذ time (n)

الـ Body بضرب عدد مرات تنفيذ كل خطوة بـ n .

C5 j = 0 رح يتنفذ مرة وحدة بس time (1 * n)

C6 j < n رح يتنفذ time (n + 1) * n

C7 s = s + a[i][j] رح يتنفذ (n * n)

C8 j ++ رح يتنفذ time (n * n)

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	1	*	C_2	=	C_2
	$(n+1)$	*	C_3	=	$(n+1)C_3 = nC_3 + C_3$
	n	*	C_4	=	nC_4
	n	*	C_5	=	nC_5
	$(n+1) * n$	*	C_6	=	$C_6 = (n^2 + n) * C_6 = n^2C_6 + nC_6$
	$n * n$	*	C_7	=	$C_7 = n^2C_7$
	$n * n$	*	C_8	=	$C_8 = n^2C_8$
	1	*	C_9	=	$C_9 = C_9$

$$\text{Running Time} = C_1 + C_2 + nC_3 + C_3 + nC_4 + nC_5 + n^2C_6 + nC_6 + n^2C_7 + n^2C_8 + C_9$$

$$= C_1 + C_2 + nC_3 + C_3 + nC_4 + nC_5 + n^2C_6 + nC_6 + n^2C_7 + n^2C_8 + C_9$$

$$= C_{10} + nC_3 + nC_4 + nC_5 + n^2C_6 + nC_6 + n^2C_7 + n^2C_8$$

$$= C_{10} + n(C_3 + C_4 + C_5 + C_6) + n^2C_6 + n^2C_7 + n^2C_8$$

$$= C_{10} + n(C_{11}) + n^2C_6 + n^2C_7 + n^2C_8$$

$$= C_{10} + n(C_{11}) + n^2(C_6 + C_7 + C_8)$$

$$= C_{10} + n(C_{11}) + n^2(C_{12})$$

$$= C_{10} + nC_{11} + n^2C_{12}$$

$$\text{Running time} = n^2C_{12} + nC_{11} + C_{10}$$

```
for ( int i=0 ; i<n ; i++ ) {
}
```

نقطة البداية a

نقطة النهاية b

عشان اعرف الـ loop كم رح تلف (i ++ و الـ Body) ، النهاية - البداية = b - a = n – 0 = n

يعني يساوي n

الشرط (i < n) بكون الناتج + 1 = 1 + n

```
for ( int i=10 ; i<n ; i++ ) {
    x = x + 1 ;
}
```

نقطة النهاية b

نقطة البداية a

النهاية - البداية = b - a = n - 10

i++ و x = x+1 رح يلفو بمقدار n – 10

i < n رح تلف بمقدار 1 + (n – 10) = n - 9

Time	*	Cost
1	*	$C_1 = C_1$
n – 9	*	$C_2 = n C_2 - 9C_2$
n – 10	*	$C_3 = n C_3 - 10C_3$
n – 10	*	$C_4 = n C_4 + 10C_4$

$$= C_1 + nC_2 - 9C_2 + nC_3 - 10C_3 + nC_4 - 10C_4$$

$$= C_5 + nC_2 + nC_3 + nC_4$$

$$= C_5 + n (C_2 + C_3 + C_4)$$

$$\text{Running time} = C_5 + nC_6$$

```
for ( int i=10 ; i<n-10 ; i++ ) { }
```

a نقطة البداية b نقطة النهاية

$$a = 10 \quad , \quad b = n - 10$$

$$b - a = (n - 10) - 10$$

$$= n - 20$$

الـ loop كم رح تلف (i ++ و الـ Body) = $n - 20$

الشرط ($i < n - 10$) = $(n - 20) + 1 = n - 19$

Running Time	=	Time	*	Cost		
		1	*	C_1	=	C_1
		$n - 20$	*	C_2	=	$nC_2 - 20C_2$
		$n - 20$	*	C_3	=	$nC_3 - 20C_3$
		$n - 19$	*	C_4	=	$nC_4 - 19C_4$

$$= C1 + nC_2 - 20C_2 + nC_3 - 20C_3 + nC_4 - 19C_4$$

$$= C_5 + nC_2 + nC_3 + nC_4$$

$$= C_5 + n (C_2 + C_3 + C_4)$$

$$\text{Running time} = C_5 + nC_6$$

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

```
for ( int i=n ; i<0 ; i-- ) { }
```

نقطة البداية a

نقطة النهاية b

$a = n \quad , \quad b = 0$

$b - a = n - 0$

$= n$

الـ loop كم رح تلف (i ++ و الـ Body) = n

الشرط (i < 0) = (n) + 1 = n + 1

بما انه الشرط فيه = ، بزيد 1 عالحالة العادية بكل جزء بالـ loop .

```
for ( int i=0 ; i<=n ; i++ ) { }
```

نقطة البداية a

نقطة النهاية b

$a = 1 \quad , \quad b = n$

$b - a = (n - 0) = n$

بدي ازيد واحد عالناتج عشان الـ = يلي جوّا الشرط .

$= n + 1$

الـ loop كم رح تلف (i ++ و الـ Body) = n + 1

الشرط (i <= n) = (n + 1) + 1 = n + 2

```
for ( int i=1 ; i<=n ; i++ ) { }
```

نقطة البداية a

نقطة النهاية b

$a = 1 \quad , \quad b = n$

$b - a = (n - 1) = n - 1$

بدي ازيد واحد عالناتج عشان الـ = يلي جوّا الشرط .

$= n - 1 + 1 = n$

الـ loop كم رح تلف (i ++ و الـ Body) = n

الشرط (i <= n) = (n) + 1 = n + 1

عندي 2 loop منفصلات

```
int x = 0 ;
```

C1 رح يتنفذ مرة وحدة بس time (1)

```
for ( int i=0 ; i<=n ; i++ ) {
    // Body1
    }
```

C2 i = 0 رح يتنفذ مرة وحدة بس time (1)

C3 Body1 رح يتنفذ time (n + 1)

C4 i ++ رح يتنفذ time (n + 1)

C5 i <= n رح يتنفذ time (n + 2)

```
for ( int j=0 ; j<n ; j++ ) {
    // Body2
    }
```

C6 j = 0 رح يتنفذ مرة وحدة بس time (1)

C7 j < n رح يتنفذ time (n + 1)

C8 Body2 رح يتنفذ time (n)

C9 j ++ رح يتنفذ time (n)

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	1	*	C_2	=	C_2
	$(n+1)$	*	C_3	=	$(n+1)C_3 = nC_3 + C_3$
	$(n+1)$	*	C_4	=	$(n+1)C_4 = nC_4 + C_4$
	$(n+2)$	*	C_5	=	$(n+2)C_5 = nC_5 + 2C_5$
	1	*	C_6	=	C_6
	$(n+1)$	*	C_7	=	$(n+1)C_7 = nC_7 + C_7$
	n	*	C_8	=	nC_8
	n	*	C_9	=	nC_9

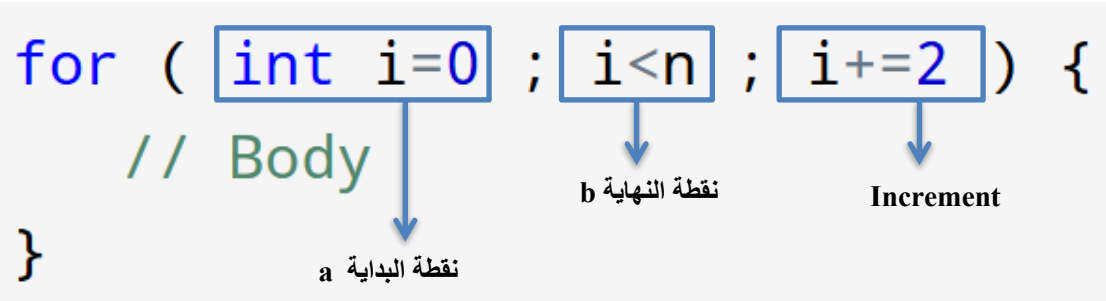
$$= C_1 + C_2 + nC_3 + C_3 + nC_4 + C_4 + nC_5 + 2C_5 + C_6 + nC_7 + C_7 + nC_8 + nC_9$$

$$= C_{10} + nC_3 + nC_4 + nC_5 + nC_7 + nC_8 + nC_9$$

$$= C_{10} + n(C_4 + C_5 + C_7 + C_8 + C_9)$$

$$= C_{10} + nC_{11}$$

Running time $= C_{10} + nC_{11}$



$a = 0 \quad , \quad b = n \quad , \quad inc = 2$

$$\frac{b-a}{inc}$$

بستخدم هاي المعادلة اذا الـ increment مش 1 .

$$(b - a) / inc = (n - 0) / 2 = n/2$$

```
for ( int i=0 ; i<n ; i+=3 ) {
    // Body
}
```

a = 0 b = n inc = 3

$$(b - a) / inc = (n - 0) / 3 = n/3$$

الـ loop كم رح تلف (i ++ و الـ Body) = n/3

الشرط (i < n) = (n/3) + 1 = n/3 + 1

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	n/3	*	C_2	=	$n/3\ C_2$
	n/3	*	C_3	=	$n/3\ C_3$
	n/3 + 1	*	C_4	=	$(n/3 + 1) C_4 = n/3C_4 + C_4$

$$= C_1 + n/3\ C_2 + n/3\ C_3 + n/3C_4 + C_4$$

$$= C_5 + n/3\ C_2 + n/3\ C_3 + n/3C_4$$

$$= C_5 + n (C_2/3 + C_3/3 + C_4/3)$$

$$= C_5 + nC_6$$

```
for ( int i=0 ; i<=n ; i+=2 ) {
    // Body
}
```

a = 0 b = n inc = 2

+ 1 عشان الـ = بالشرط .

بما انه الشرط فيه = ، بزيد 1 عالحالة العادية بالـ loop على قيمة b .

$$(b - a) / inc = (n + 1 - 0) / 2 = (n + 1) /2$$

الـ loop كم رح تلف (i ++ و الـ Body) = (n + 1) / 2

الشرط (i <= n) = (n + 1) / 2 + 1

```
for ( int i=0 ; i<n/2 ; i+=2 ) {
    // Body
}
```

a = 0

b = n / 2

inc = 2

(b – a) / inc = (n/2 – 0) / 2 = (n/2) /2 = n / 4

الـ loop كم رح تلف (i ++ و الـ Body) = n / 4

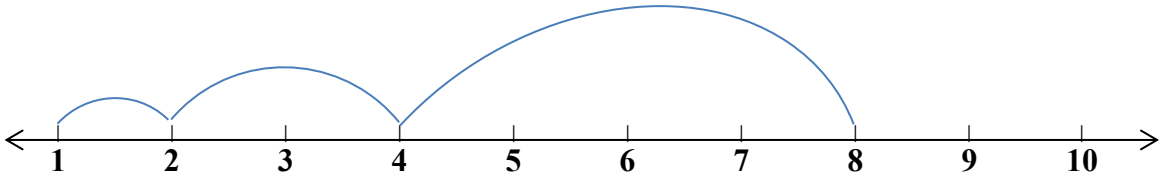
الشرط (i <= n) = (n / 4) + 1

```
for ( int i=1 ; i<n ; i*=2 ) {
    // Body
}
```

لما الـ increment مش خطي يكون ضرب او قسمة

لنفرض n = 4

	Condition	Body	Increment
i = 1	i < n		i * 1 = 2 ✓
i = 2	✓	✓	1 * 2 = 2 ✓
i = 4	✓	✓	2 * 2 = 4 ✓
i = 8	✓ بدخل لـ الشرط عشان التحقق بس بفشل	ما بكمل لانه الشرط ما كان صح	ما بكمل لانه الشرط ما كان صح



الـ loop بمشي بمقدار تزايد مش خطي log

$$\text{Log}_{2}(n)$$

```
for ( int i=1 ; i<n ; i*=2 ) {
    // Body
}
```

$$\text{Log}_{2}(n)$$

$$\frac{b}{a} = \frac{n}{1} = n$$

يلي بحدد الترتيب في القسمة ($\frac{b}{a}$) أو ($\frac{a}{b}$) هو : هل i بتكبر أو بتصغر مع كل تكرار بالـ loop

```
for ( int i=a ; i<b ; i *=num ) { ... } // b/a
for ( int i=a ; i>b ; i /=num ) { ... } // a/b
```

```
for ( int i=1 ; i<n ; i*=2 ) {
    // Body
}
```

a = 1 b = n

الـ loop كم رح تلف (i*=2 و الـ Body) = $\log_2(n)$

الشرط (i < n) = $\log_2(n) + 1$

```
for ( int i=1 ; i<n ; i*=2 ) {
    // Body
}
```

time (1) رح يتنفذ مرة وحدة بس i = 1 C1

time ($\log_2(n)$) رح يتنفذ Body C4

time ($\log_2(n)$) رح يتنفذ i*=2 C3

time ($\log_2(n) + 1$) رح يتنفذ i < n C2

Running Time	=	Time	*	Cost		
		1	*	C_1	=	C_1
		$\log_2(n) + 1$	*	C_2	=	$\log_2(n)\,C_2 + C_2$
		$\log_2(n)$	*	C_3	=	$\log_2(n)\,C_3$
		$\log_2(n)$	*	C_4	=	$\log_2(n)\,C_4$

$$= C_1 + \log_2(n)\,C_2 + C_2 + \log_2(n)\,C_3 + \log_2(n)\,C_4$$

$$= C_5 + \log_2(n)\,C_2 + \log_2(n)\,C_3 + \log_2(n)\,C_4$$

$$= C_5 + \log_2(n)(C_2 + C_3 + C_4)$$

$$= C_5 + \log2(n)\,C_6$$

Running time $= C_5 + \log2(n)\,C_6$

```
for ( int i=n/2 ; i>1 ; i/=2 ) {
    // Body
}
```

a = n/2

b = 1

$$\frac{a}{b} = \frac{n/2}{1} = n/2$$

$$\text{Log}_2 (n/2)$$

```
for ( int i=0 ; i<n ; i++ ) {
    for ( int j=1 ; j<n ; j*=2 ) {
    // Body
    }
}
```

time (n) رح يتنفذ C3

time (1) رح يتنفذ مرة وحدة بس C1

time (n+ 1) رح يتنفذ C2

Body * n

time (1) رح يتنفذ مرة وحدة بس j = 1 C4

time ($\log_2$ (n) + 1) رح يتنفذ j < n C5

time ($\log_2$ (n)) رح يتنفذ Body C6

time ($\log_2$ (n)) رح يتنفذ j*=2 C7

الـ Body بضرب عدد مرات تنفيذ كل خطوة بـ n .

time (1 * n) رح يتنفذ مرة وحدة بس j = 1	C4
time (($\log_2$ (n) + 1) *n) رح يتنفذ j < n	C5
time (($\log_2$ (n)) * n) رح يتنفذ Body	C6
time (($\log_2$ (n)) * n) رح يتنفذ j*=2	C7

Running Time = Time	*	Cost		
1	*	C_1	=	C_1
n + 1	*	C_2	=	$nC_2 + C_2$
(n)	*	C_3	=	nC_3
(1 * n)	*	C_4	=	nC_4
($\log_2$ (n) + 1) * n	*	C_5	=	(log2 (n) * n + n) C_5
				= log2 (n) * n $C_5 + nC_5$
($\log_2$ (n)) * n	*	C_6	=	$\log_2$ (n) * n C_6
($\log_2$ (n)) * n	*	C_7	=	$\log_2$ (n) * n C_7

$$= C_1 + nC_2 + C_2 + nC_3 + nC_4 + \log_2(n) * n\, C_6 + nC_5 + \log_2(n) * n\, C_6 + \log_2(n) * n\, C_7$$

$$= C_8 + nC_2 + nC_3 + nC_4 + \log_2(n) * n\, C_5 + nC_5 + \log_2(n) * n\, C_6 + \log_2(n) * n\, C_7$$

$$= C_8 + n(C_2 + C_3 + C_4 + C_5) + \log_2(n) * n\, C_5 + \log_2(n) * n\, C_6 + \log_2(n) * n\, C_7$$

$$= C_8 + nC_9 + \log_2(n) * n\,(C_5 + C_6 + C_7)$$

$$= C_8 + nC_9 + n \log_2(n)\, C_{10}$$

$$\text{Running time} = C_8 + nC_9 + n \log_2(n)\, C_{10}$$

```
for ( int i=1 ; i<=n ; i++ ) {

    for ( int j=1 ; j<=i ; j++ ) {
        System.out.print(i*j);
    }

}
```

Body * n

الـ for الأولى الخارجية :

```
for ( int i=1 ; i<=n ; i++ )
```

a = 1 b = n inc = 1

+ 1 عشان الـ = بالشرط .

بما انه الشرط فيه = ، بزيد 1 عالحالة العادية بالـ loop على قيمة b .

(b – a) / inc = (n + 1 – 1) / 1 = (n) /1 = n

الـ loop كم رح تلف (i ++ و الـ Body) = n

الشرط (i <= n) = (n + 1)

```
for ( int i=1 ; i<=n ; i++ )
```

C1 i = 1 رح يتنفذ مرة وحدة بس time (1)

C3 i++ رح يتنفذ time (n)

C2 i <= n رح يتنفذ time (n + 1)

الـ for الداخلية :

```
for ( int j=1 ; j<=i ; j++ )
```

بالشرط فيه i وانا ما بعرف كم قيمتها تحديداً ، لانها بتتغير حسب الـ loop الخارجية

لنفرض n = 5

i رح تمشي من 1 الى 5 (6 الشرط بفشل)

بس تكون i = 1 الـ Body الداخلية رح تلف مرة 1 ، وبالشرط رح تلف 1 + 1

بس تكون i = 2 الـ Body الداخلية رح تلف مرتين 2 ، وبالشرط رح تلف 1 + 2

بس تكون i = 3 الـ Body الداخلية رح تلف 3 مرات ، وبالشرط رح تلف 1 + 3

بس تكون i = 4 الـ Body الداخلية رح تلف 4 مرات ، وبالشرط رح تلف 1 + 4

بس تكون i = 5 الـ Body الداخلية رح تلف 5 مرات ، وبالشرط رح تلف 1 + 5

هيك خلص الشرط وقف ، ولو بدي احسب عدد مرات تنفيذ الـ loop الداخلية رح تكون

1 + 2 + 3 + 4 + 5 = 15

الـ loop كم رح تلف (j ++ و الـ Body) = $\sum_{1}^{n} i = \frac{n(n+1)}{2}$

الشرط (j<= i) = (15 + 5) ، مش +1 لانه كل مرة دخلت عالـ loop زدت 1 كم مرة دخلت ؟ 5 مرات

الشرط (j<= i) = $\sum_{1}^{n} i = \frac{n(n+1)}{2} + n$

هاد القانون من حساب تكرار الـ Two loop ، يعني كأنه ضربته بـ n فـ ما بلزم اضرب الـ Body بـ n غير بـ اول خطوة int j = 1 لانها مش داخلة بقانون المتتالية .

```
for ( int j=1 ; j<=i ; j++ ) {
    System.out.print(i * j);
}
```

$1 * n \quad C_4$

$\frac{n(n+1)}{2} + n \quad C_5$

$\frac{n(n+1)}{2} \quad C_6$

$\frac{n(n+1)}{2} \quad C_7$

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	n	*	C_2	=	nC_2
	$n + 1$	*	C_3	=	$nC_3 + nC_3$
	$1 * n$	*	C_4	=	$n\,C_4$
	$(n (n + 1) / 2 + n)$	*	C_5	=	$nC_5 + (n (n + 1) / 2 + n) C_5$
	$(n (n + 1) / 2)$	*	C_6	=	$(n (n + 1) / 2) C_6$
	$(n (n + 1) / 2)$	*	C_7	=	$(n (n + 1) / 2) C_7$

$$= C_1 + C_2 + nC_3 + nC_4 + nC_5 + (n (n + 1) / 2 + n) C_5 + (n (n + 1) / 2) C_6 + (n (n + 1) / 2) C_7$$

$$= C_1 + C_2 + nC_3 + nC_4 + nC_5 + ((n^2 + n) / 2 + n) C_5 + ((n^2 + n) / 2) C_6 + ((n^2 + n) / 2) C_7$$

$$= C_8 + n (C_3 + C_4 + C_5) + ((n^2 + n) / 2 + n) C_5 + (n^2 + n) (C_7/2 + C_6/2)$$

$$= C_8 + n\, C_9 + ((n^2 + n) / 2 + n) C_5 + (n^2 + n) C_{10}$$

$$= C_8 + n\,C_9 + (\,(\,n^2 + n)\,/\,2 + n\,)\,C_5 + (\,n^2 + n)\,C_{10}$$

$$= C_8 + n\,C_9 + n^2\,C_5/2 + n\,C_5/2 + n\,C_5 + n^2\,C_{10} + n\,C_{10}$$

$$= C_8 + n\,(\,C_9 + C_5/2 + C_5 + C_{10}\,) + n^2\,(\,C_5/2\;C_{10}\,)$$

$$= C_8 + n\,C_{11} + n^2\,C_{12}$$

Running time $= C_8 + n\,C_{11} + n^2\,C_{12}$

```
for ( int i=0 ; i<n ; i++ ) {

    for ( int j=1 ; j<i ; j++ ) {
    // body
    }
}
```

$$\sum_{1}^{n-1} i = \frac{n(\,n-1\,)}{2}$$

(معلومة مش مطلوب معنا)

```
for ( int i=0 ; i<n ; i*=2 ) {

    for ( int j=0 ; j<n ; j++ ) {
    // body
    }
}
```

i قيمتها = 0 ، بعد ما يتحقق الشرط i < n ويجي يكمل الـ loop رح يلاقي i *=2

0 * 2 = 0 ، قيمة i ما رح تتغير رح تضل صفر والـ loop رح يكون ∞

	Condition	Body	Increment
i = 0	i < n		0 * 0 = 0 ✓
i = 0	✓	✓	0 * 0 = 0 ✓
i = 0	✓	✓	0 * 0 = 0 ✓
i = 0 رح تضل قيمتها ثابتة	∞ الشرط يتحقق ويتنفذ باستمرار	∞	∞

a = 1

b = n

inc = 5

Body * $\log_5$ (n)

```
for ( int i=1 ; i<n ; i*=5 ) {

    for ( int j=0 ; j<n ; j++ ) {
    // body
    }
}
```

time ($\log_5$ (n) + 1) رح يتنفذ C2

time (1) رح يتنفذ مرة وحدة بس C1

time $\log_5$ (n) رح يتنفذ C3

$$\text{Log}_{5}(n)$$

inc = 5

$$\frac{b}{a} = \frac{n}{1} = n$$

الـ loop كم رح تلف (i*=5) و الـ Body) = $\log_5$ (n)

الشرط (i < n) = $\log_5$ (n) + 1

```
for ( int j=0 ; j<n ; j++ ) {
// body
}
```

time (1 * $\log_5$ (n)) رح يتنفذ مرة وحدة بس j = 0 C4

time (n * $\log_5$ (n)) رح يتنفذ j ++ C5

time ((n + 1) * $\log_5$ (n)) رح يتنفذ j<n C6

time (n * $\log_5$ (n)) رح يتنفذ Body C7

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	$\log_5(n) + 1$	*	C_2	=	$\log_5(n) C_2 + C_2$
	$\log_5(n)$	*	C_3	=	$\log_5(n) C_3$
	$1 * \log_5(n)$	*	C_4	=	$\log_5(n) C_4$
	$n * \log_5(n)$	*	C_5	=	$\log_5(n) * n\, C_5$
	$n * \log_5(n)$	*	C_6	=	$\log_5(n) * n\, C_6$
	$(n+1) * \log_5(n)$	*	C_7	=	$(n+1) * \log_5(n) C_7$
					$= \log_5(n) * n\, C_7 + \log_5(n) C_7$

$$= C_1 + \log_5(n) C_2 + C_2 + \log_5(n) C_3 + \log_5(n) C_4 + \log_5(n) * n\, C_5 + \log_5(n) * n\, C_6 + \log_5(n) * n\, C_7 + \log_5(n) C_7$$

$$= C_8 + \log_5(n) (C_3 + C_4 + C_7) + \log_5(n) * n\, (C_5 + C_6 + C_7)$$

$$= C_8 + \log_5(n) C_9 + \log_5(n) * n\, C_{10}$$

$$\text{Running time} = C_8 + \log_5(n) C_9 + \log_5(n) * n\, C_{10}$$

```
for ( int i=0 ; i<n ; i++ ) {

    for ( int j=0 ; j<n ; j++ ) {

        for ( int z=0 ; z<n ; z++ ) {
          // Body
        }

    }

}
```

Body1 * n

Body2 * n

الـ for الأولى (i) :

```
for ( int i=0 ; i<n ; i++ )
```

C1 رح يتنفذ مرة وحدة بس time (1)

C2 رح يتنفذ time (n)

C3 رح يتنفذ time (n+ 1)

الـ for الثانية (j) :

```
for ( int j=0 ; j<n ; j++ )
```

C4 رح يتنفذ مرة وحدة بس time (1 * n)

C6 رح يتنفذ time (n * n)

C5 رح يتنفذ time ((n+ 1) * n)

الـ for الثالثة (z) :

```
for ( int z=0 ; z<n ; z++ )
```

C8 رح يتنفذ مرة وحدة بس time (1 * n^2)

C10 رح يتنفذ time (n * n^2)

C9 رح يتنفذ time ((n+ 1) * n^2)

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	n	*	C_2	=	$n\,C_2$
	$(n+1)$	*	C_3	=	$n\,C_3 + C_3$
	$1 * n$	*	C_4	=	$n\,C_4$
	$(n+1) * n$	*	C_5	=	$n^2\,C_5 + n\,C_5$
	$n * n$	*	C_6	=	$n^2\,C_6$
	$1 * n^2$	*	C_7	=	$n^2\,C_7$
	$(n+1) * n^2$	*	C_8	=	$n^3\,C_8 + n^2\,C_8$
	$n * n^2$	*	C_9	=	$n^3\,C_9$

$$= C_1 + n\,C_2 + n\,C_3 + C_3 + n\,C_4 + n^2\,C_5 + n\,C_5 + n^2\,C_6 + n^2\,C_7 + n^3\,C_8 + n^2\,C_8 + n^3\,C_9$$

$$= C_{11} + n\,(C_2 + C_3 + C_4 + C_5) + n^2\,(C_5 + C_6 + C_7 + C_8) + n^3\,(C_8 + C_9)$$

$$= C_{11} + n\,C_{12} + n^2\,C_{13} + n^3\,C_{14}$$

Running time $= C_{11} + n\,C_{12} + n^2\,C_{13} + n^3\,C_{14}$

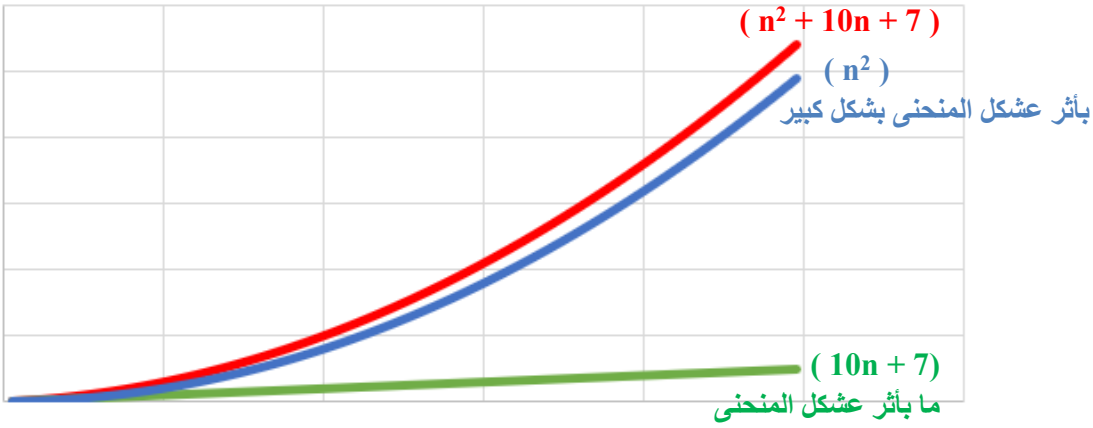
"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Asymptotic Notation

$RT_1 = n^2 + 10n + 7$

جربت المعادلة على Excel عشان ارسم منحنى مع القيم .

	$n^2+10n+7$	$10n+7$	n^2
1	18	17	1
2	31	27	4
3	46	37	9
4	63	47	16
5	82	57	25
6	103	67	36
7	126	77	49
8	151	87	64
9	178	97	81
10	207	107	100
11	238	117	121
12	271	127	144
13	306	137	169
14	343	147	196
15	382	157	225



Big O notation : عشان اعرف الـ worst case ، في حد بالـRunning Time الـRunning Time كله بيعتمد عليه ، لما بدي Big O لمعادلة ، يعني سَقِف منحنى ممنوع يكون منحنى تاني بيتجاوزه وهو اعلى اشي ، يعني اكبر Power .

$$F(n) = O(g(n))$$

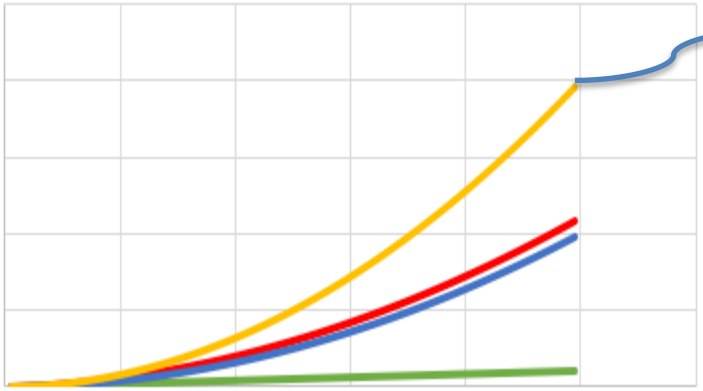
n2 هي اعلى سقف بس بشرطين :

$F(n) = O(g(n))$

$F(n) \leq C_1 * g(n)$

الشرط الأول بضربها بـ constant مثلاً = 2 .

	$n^2+10n+7$	$10n+7$	n^2	$n^2 * 2$
1	18	17	1	2
2	31	27	4	8
3	46	37	9	18
4	63	47	16	32
5	82	57	25	50
6	103	67	36	72
7	126	77	49	98
8	151	87	64	128
9	178	97	81	162
10	207	107	100	200
11	238	117	121	242
12	271	127	144	288
13	306	137	169	338
14	343	147	196	392
15	382	157	225	450



n^2 بعد ما ضربتها بـ constant صارت اعلى سقف بالمنحنى ما في اي منحنى رح يتجاوزه فهاي أسوء حالة ممكن الـ algorithm يوصلّلها وهي الـ worst case .

الشرط الثاني انه رح أوصل لنقطة مُعينة من n مثلاً رح اسميها n_0 تنتمي لـ قيم n ، ببلش المنحنى من وبعد هالنقطة يكون هو السقف ، ومستحيل بعد هالنقطة الـ Running Time يتجاوز الـ Big O (n) .

	n^2+10n+7		n^2 * 2
1	18	>	2
2	31	>	8
3	46	>	18
4	63	>	32
5	82	>	50
6	103	>	72
7	126	>	98
8	151	>	128
9	178	>	162
10	207	>	200
11	238	<	242
12	271	<	288
13	306	<	338
14	343	<	392
15	382	<	450
16	423	<	512
17	466	<	578
18	511	<	648

عند n = 11 كل قيم n^2 اكبر من الـ Running Time .

Big Ω notation : عشان اعرف الـ best case ، في حد بالـ Running Timeالـ Running Time كله بيعتمد عليه ، لما بدي Big Ω لمعادلة ، يعني أرضية المنحنى ممنوع يكون منحنى تاني بنزل تحته وهو أقل اشي ، يعني أصغر Power .

- Best Case الاقي العنصر من أول محاولة
- Worst Case العنصر يكون آخر اشي أو مش موجود

بالـ Worst Case بمسك أعلى Power السقف ، بالـ Best Case بمسك أقل Power الأرضية .

Big Θ notation : عشان اعرف الـ average case ، في خط بالـ Running Time بيمثل الأداء المتوقع بالعادة ، يعني مش أعلى اشي مثل الـ worst Case ولا أقل اشي مثل الـ best Case ، هو المتوسط يلي غالباً الخوارزمية بتمشي عليه وهي أكثر دقة .

$$C_2\, g(n) \le f(n) \le C_1\, g(n)$$

```
for i←0 to n step 1 do
    print i * i
end for
```

time (1) رح يتنفذ مرة وحدة C1

time (n) رح يتنفذ C2

time (n+1) رح يتنفذ C3

time (n) رح يتنفذ C4

a = 0 , b = n , inc = 1

$$\frac{b-a}{inc}$$

(b – a) / inc = (n – 0) / 1 = n

الـ loop رح تلف (step 1 و الـ Body) = n

الشرط = (n + 1)

Running Time = Time * Cost

$1 * C_1 = C_1$

$(n) * C_2 = nC_2$

$(n+1) * C_3 = nC_3 + C_3$

$(n) * C_4 = nC_4$

$= C_1 + nC_2 + nC_3 + C_3 + nC_4$

$= C_5 + n(C_2 + C_3 + C_4)$

$= C_5 + nC_6$

Big O ($C_5 + nC_6$)

بدون constant وباخذ بس أكبر power

Big O (n)

Example :

$RT_1 = n^2 + 10n + 7$
Big O (n^2) // اكبر قوة + بدون ثوابت

$RT_2 = 100n^2 + 10n + 7$
Big O (n^2)

$RT_3 = n^2/ 2 + 30n$
Big O (n^2)

كل ما زادت n كل ما زاد الوقت والـ Time Complexity .

```
for i←0 to √n step 1 do
    √n * √i    print i
end for
```

C1 رح يتنفذ مرة وحدة بس time (1)

C2 رح يتنفذ time (√n)

C3 رح يتنفذ time (√ n+ 1)

C4 رح يتنفذ time (√n)

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	√n	*	C_2	=	√n C_2
	(√n + 1)	*	C_3	=	√n C_3 + C_3
	√n	*	C_4	=	√n C_4

$$= C_1 + \sqrt{n}\, C_2 + \sqrt{n}\, C_3 + C_3 + \sqrt{n}\, C_4$$

$$= C_5 + \sqrt{n}\,(C_2 + C_3 + C_4)$$

$$= C_5 + \sqrt{n}\, C_6$$

$$\text{Big O}(\sqrt{n}) = O(n^{1/2})$$

```
for i←0 to n/3 step 3 do

    for j←0 to n/3 step 2 do
        print i * j
    end for

end for
```

Body * n/9

الـ for الخارجية :

$$a = 0 \quad , \quad b = n/3 \quad , \quad inc = 3$$

$$\frac{b-a}{inc}$$

$$(b - a) / inc = (n/3 - 0) / 3 = n/3/3 = n/9$$

الـ loop رح تلف (step 3 و الـ Body) = n/9

الشرط = (n/9 + 1)

```
for i←0 to n/3 step 3 do
```

C1 رح يتنفذ مرة وحدة بس time (1)

C2 رح يتنفذ time (n/9)

C3 رح يتنفذ time (n/9 + 1)

الـ for الداخلية :

$$a = 0 \quad , \quad b = n/3 \quad , \quad inc = 2$$

$$\frac{b-a}{inc}$$

$$(b - a) / inc = (n/3 - 0) / 2 = n/3/2 = n/6$$

الـ Body كله * n/9

الـ loop رح تلف (step 2 و الـ Body) = (n/9) * (n/6) = ($n^2/54$)

الشرط = (n + 1) = (n/9) * (n/6 + 1) = (($n^2/54$) + (n/9))

```
for j←0 to n/3 step 2 do
```

time ($n^2/54$) رح يتنفذ C6

time (($n^2/54$) + (n/9)) رح يتنفذ C5

time (1 * n/9) رح يتنفذ مرة وحدة بس C4

الـ Body

```
print i * j
```

time ($n^2/54$) رح يتنفذ C7

Running Time =	Time	*	Cost		
	1	*	C_1	=	C_1
	$(n/9)$	*	C_2	=	$(n/9)C_2$
	$(n/9 + 1)$	*	C_3	=	$(n/9)C_3 + C_3$
	$(1 * n/9)$	*	C_4	=	$(n/9)C_4$
	$((n/9) + (n^2/54))$	*	C_5	=	$(n/9)C_5 + (n^2/54)C_5$
	$(n^2/54)$	*	C_6	=	$(n^2/54)C_6$
	$(n^2/54)$	*	C_7	=	$(n^2/54)C_7$

$$= C_1 + (n/9)C_2 + (n/9)C_3 + C_3 + (n/9)C_4 + (n/9)C_5 + (n^2/54)C_5 + (n^2/54)C_6 + (n^2/54)C_7$$

$$= C_8 + (n)(C_2/9 + C_3/9 + C_4/9 + C_5/9) + (n^2)(C_5/54 + C_6/54 + C_7/54)$$

$$\text{Running time} = C_8 + nC_9 + n^2C_{10}$$

Big O ($C_8 + nC_9 + n^2C_{10}$) // constant بدون الـ

Big O ($n + n^2$) // power أكبر

Big O (n^2)

```
for i←0 to 10000 step 1 do
```

time (1) رح يتنفذ مرة وحدة بس C1

time (10000) رح يتنفذ C2

time (10001) رح يتنفذ C3

```
    for j←0 to n^2  step 1 do
        print i * j
    end for
```

Body * 10000

```
end for
```

الـ for الداخلية :

$a = 0 \quad , \quad b = n^2 \quad , \quad inc = 1$

$$\frac{b-a}{inc}$$

$(b - a) / inc = (n^2 - 0) / 1 = n^2$

الـ Body كله * 10000

الـ loop رح تلف (step 2 و الـ Body) = $10000 * (n^2)$

الشرط = $10000 * (n^2 + 1)$

```
for j←0 to n^2 step 1 do
```

C6 رح يتنفذ time $10000 * (n^2)$

C5 رح يتنفذ time $10000 * (n^2 + 1)$

C4 رح يتنفذ مرة وحدة بس time $(1 * 10000)$

```
print i * j
```

C7 رح يتنفذ time $10000 * (n^2)$

Running Time	=	Time	*	Cost		
		1	*	C_1	=	C_1
		10001	*	C_2	=	$10001C_2$
		10000	*	C_3	=	$10000C_3$
		$1 * 10000$	*	C_4	=	$10000C_4$
		$(n^2 + 1) * 10000$	*	C_5	=	$(n^2)*10000C_5 + 10000C_5$
		$(n^2) * 10000$	*	C_6	=	$(n^2)*10000C_6$
		$(n^2) * 10000$	*	C_7	=	$(n^2)*10000C_7$

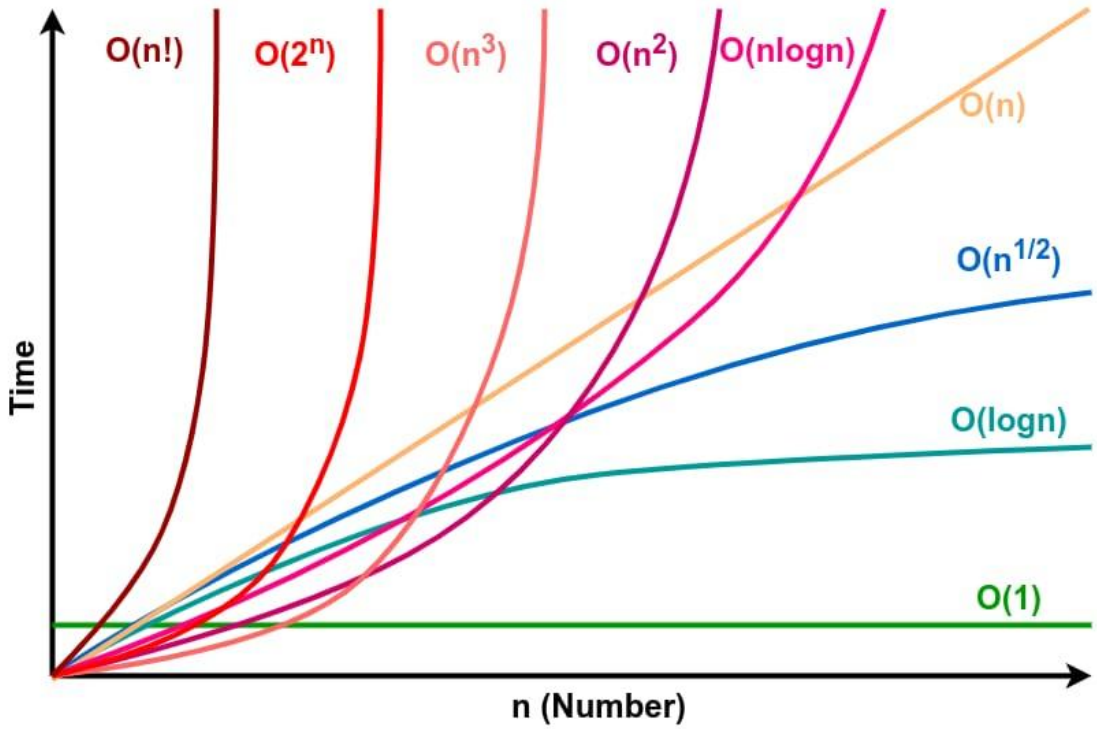
$$= C_1 + 10001C_2 + 10000C_3 + 10000C_4 + (n^2)*10000C_5 + 10000C_5 + (n^2)*10000C_6 + (n^2)*10000C_7$$

$$= C_8 + n^2 (10000C_5 + 10000C_6 + 10000C_7)$$

$$= C_8 + n^2C_9$$

Big O ($C_8 + n^2C_9$)

Big O (n^2)



معظم الـ time بكون 80% لـ input , output و 20% لـ speed .

Example :

لحساب مجموع الأرقام من 1 إلى n ، الطريقة يلي رح تخطر عبالي for loop المعتادة :

```
int sum = 0;

for ( int i=1 ; i<n ; i++ ) {
    sum += i ;
}
```

Big O (n)

طريقة اسهل و Big O اقل لحساب المجموع باستخدام معادلة :

sum = n * (n+1) / 2

n = 5 لو كانت

Sum = 5 * (5+1) / 2 = 5 * 6 / 2 = 30/2 = 15

Sum = 1 + 2 + 3 + 4 + 5 = 15

```
int sum = n * (n + 1) / 2 ;
```

وبطبع ناتج sum .

Big O (1)

- O (1) : تبقى العملية ثابتة مهما كان عدد المدخلات .
- O (n) : تزداد المدة الزمنية كلما زاد عدد العمليات .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Example :

```
for i←0 to n/2 step 1 do

    for j ← 1 to n  step *2 do

        print i * j
    end for

end for
```

Body * n/2

الـ for الخارجية :

$$a = 0 \quad , \quad b = n/2 \quad , \quad inc = 1$$

$$\frac{b-a}{inc}$$

$$(b - a) / inc = (n/2 - 0) / 1 = n/2$$

الـ loop رح تلف (step 1 و الـ Body) = n/2

الشرط = (n/2 + 1)

```
for i←0 to n/2 step 1 do
```

C1 رح يتنفذ مرة وحدة بس time (1)

C2 رح يتنفذ time (n/2)

C3 رح يتنفذ time (n/2 + 1)

الـ for الداخلية :

$a = 0 \quad , \quad b = n \quad , \quad inc = 2$

الـ inc فيه ضرب يعني log

$$\frac{b}{a} = \frac{n}{1} \; n$$

$Log_2 (n)$

الـ Body كله * n/2

الـ loop رح تلف (step 2 و الـ Body) = $Log_2 (n) * (n/2)$

الشرط = $Log_2 (n + 1) * (n/2)$

```
for j ←1 to n step *2 do
```

C6 رح يتنفذ time ($Log_2 (n) * (n/2)$)

C5 رح يتنفذ time ($Log_2 (n+1) * (n/2)$)

C4 رح يتنفذ مرة وحدة بس time ($1 * n/2$)

الـ Body

```
print i * j
```

C7 رح يتنفذ time ($Log_2 (n) * (n/2)$)

Running Time = Time	*	Cost		
1	*	C_1	=	C_1
(n/2)	*	C_2	=	(n/2)C_2
(n/2 + 1)	*	C_3	=	(n/2)C_3 + C_3
(1 * n/2)	*	C_4	=	(n/2)C_4
($\log_2$ (n+1) * (n/2))	*	C_5	=	$\log_2$ (n+1)C_5 + (n/2)C_5
($\log_2$ (n) * (n/2))	*	C_6	=	($\log_2$ (n) * (n/2))C_6
($\log_2$ (n) * (n/2))	*	C_7	=	($\log_2$ (n) * (n/2))C_7

$$= C_1 + (n/2)C_2 + (n/2)C_3 + C_3 + (n/2)C_4 + \log_2 (n+1)C_5 + (n/2)C_5 + (\log_2 (n) * (n/2))C_6 + (\log_2 (n) * (n/2))C_7$$

$$= C_8 + n (C_2/2 + C_3/2 + C_4/2 + C_5/2) + (\log_2 (n) * (n)) (C_6/2 + C_7/2)$$

$$= C_8 + nC_9 + (\log_2 (n) * (n))C_{10}$$

$$= C_8 + nC_9 + n \log_2 n \, C_{10}$$

Big O ($C_8 + nC_9 + n \log_2 n \, C_{10}$)

بدون الـ constant و أكبر power

= Big O ($n \log_2 n$)

ترتيب الـ solution من الأفضل الى الأسوء :

- O(c)
- O(log)
- O(√n)
- O(n)
- O(n^2)
- O(x^n)
- O(n!)

اذا كان الـ Running Time معنا ، كيف بنحسب الـ Big Ω , Big O , Big Θ ؟

Running Time = $2n^2 + 100$

Big O (?)

- F (n) هي نفسها الـ Running Time .
- باخذ اكبر term بالـ Running Time = $2n^2$ ، بعدها عشان اكتب term جديد بعطيه قيمة أعلى من 2 لانه الـ O أعلى سقف ، بكون الـ term الجديد $3n^2$.
- C_1 لازم تكون اكبر من 2 لانه احنا بدنا $C_1 * n^2$ تكون اكبر من $2n^2$.

$$F(n) \le C_1\, g(n)$$

$$F(n) \le C_1 * n^2$$

$$C_1 > 2$$

$C_1 = 3$ (الافضل اول قيمة اكبر من 2)

$$\text{Big O}(3n^2) = n^2$$

Big Ω (?)

- باخذ اكبر term بالـ Running Time = $2n^2$ ، بعدها عشان اكتب term جديد بعطيه قيمة اقل من 2 لانه الـ Ω أقل سقف بس الـ O أعلى سقف ، بكون الـ term الجديد $1n^2$

$$F(n) \le C_2\, g(n) \le C_1\, g(n)$$

$$2n^2 \le C_2\, n2 \le 3n^2$$

$$2 \le C_2 \le 3$$

$$C_2 = 1$$

$$\text{Big } \Omega(1\, n^2) = n^2$$

	Running Time	Ω	O
n	$2n^2$	$1 * n^2$	$3 * n^2$
1	2	1	3
2	8	4	12
3	18	9	27

Big Θ (?)

$$C_2\, g(n) \le C_3\, g(n) \le C_1\, g(n)$$

$$1n^2 \le C_3 * n^2 \le 3n^2$$

$$1 \le C_3 \le 3$$

$$C_3 = 2$$

$$\text{Big } \Theta(2n^2) = n^2$$

هاد الحل بزبط بس معنا الـ Running Time .

Example :

```
k = 0 ;
for ( int i=1 ; i<n ; i++ ) {

    for ( int j=0 ; k!=0 ; j++ ) {

    }
}
```

الشرط ما رح يتحقق فالـ loop الداخلية ما رح تلف

Big O (n)

Big Ω ما بقدر احكم عليها بس لو كانت k = n وبدي احسب Big Ω = n^2

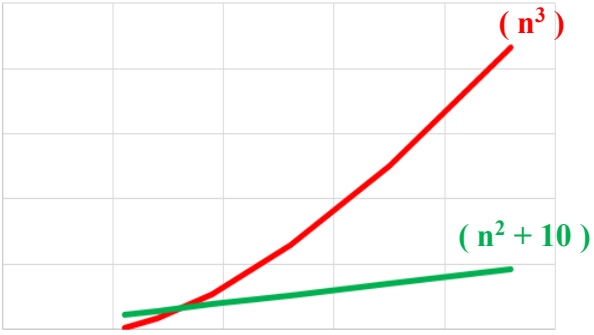
- الـ Big O بحسبه عادي من الـ Running Time .
- الـ Big Ω ما بقدر احسبه بدون وصف او كود .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Example :

أثبت أن $O(n^3)$ هو سقف الـ $\text{Running Time} = n^2 + 10$

n	$n^2 + 10$	n^3
1	11	1
2	14	8
3	19	27
4	26	64
5	35	125
6	46	216



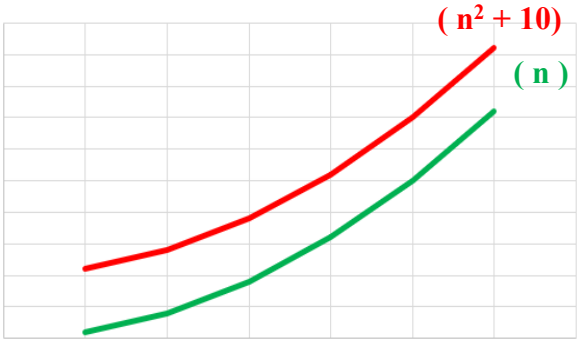
n^3 اعلى سقف

عند $n = 2$ كل قيم n^3 اكبر من الـ Running Time .

Example :

أثبت أن $\Omega(n)$ هو ليس سقف الـ $\text{Running Time} = n^2 + 10$

n	$n^2 + 10$
1	11
2	14
3	19
4	26
5	35
6	46

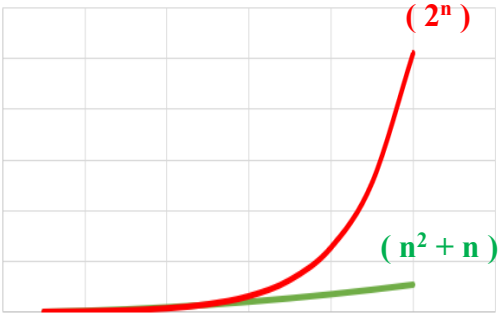


n اقل من سقف $n^2 + 10$

Example :

أثبت أن $O(2^n)$ هو سقف الـ $F(n) = n^2 + n$

n	$n^2 + n$	2^n
1	2	2
2	6	4
3	12	8
4	20	16
5	30	32
6	42	64



2^n اعلى سقف

عند $n = 5$ كل قيم 2^n اكبر من الـ Running Time .

Example :

أثبت أن $O(n^2)$ هو سقف الـ $F(n) = n^2 + n$

رح احتاج اضربها بـ C ليش ؟ لانه n^2 متساوية مع اكبر power بالـ $F(n)$.

بنفس الطريقة يلي كنا نسحب فيها C

$$F(n) \le C_1\, g(n)$$

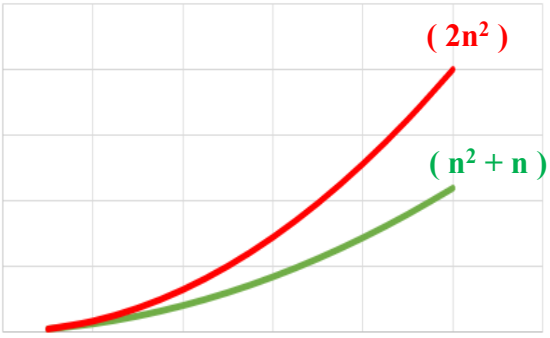
$$F(n) \le C_1 * n^2$$

$$n^2 \le C_1 * n^2$$

$$C_1 > 1$$

$C_1 = 2$ (اول قيمة اكبر من 1)

n	$n^2 + n$	$2n^2$
1	2	2
2	6	8
3	12	18
4	20	32
5	30	50



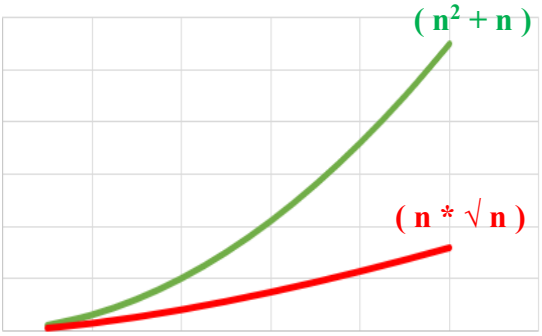
$2n^2$ اعلى سقف

عند $n = 4$ كل قيم $2n^2$ اكبر من الـ Running Time .

Example :

أثبت أن $O (n * \sqrt{n})$ لـ $F (n) = n^2 + n$

n	$n^2 + n$	$n * \sqrt{n}$
1	2	1
2	6	2.82
3	12	5.19
4	20	8
5	30	11.18



لو كملنا بـ n رح تضل اقل من الـ Running Time يعني $n * \sqrt{n}$ ما بتحقق انها تكون سقف .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Sorting Algorithm

- Bubble Sort
- Selection Sort
- Insertion Sort
- Heap Sort

Selection Sort

هاد واحد من أبسط خوارزميات الترتيب (Sorting Algorithms) ، بمرّ على الـ array أو أكثر من مرّة وبكل مرة بدور على اصغر عنصر او اكبر حسب المطلوب ترتيب تصاعدي او تنازلي ، مثلاً الأكبر بعد ما الاقيه ببدله (swap) مع اخر عنصر بالجزء الغير مرتب بالـ array و بكمل لحد ما تخلص العناصر كلها .

عدد العناصر كلها بالـ array n = 5 بلش ادور من n=0 لعند n-1 = 4 وبدور على اكبر عنصر موجود بالـ array هو الـ 9 لهيك ببدله مع اخر عنصر موجود بالـarray

3	9	6	1	8
n=0	n=1	n=2	n=3	n=4

كنا ندور من العنصر 0 لـ n-1 ، بس ابلش بالترتيب رح ادور من 0 لـ n-2 بعدها n-3 ....

3	8	6	1	9

→

3	1	6	8	9

هسا بلش ادور من 0 لـ n-3

3	1	6	8	9

→

3	1	6	8	9

رح تضل مكانها الـ 6 واكمل من 0 لـ n-4

3	1	6	8	9

→

1	3	6	8	9

رح اضل امشي بنفس الخطوات عشان تكون الـ array بالنهاية مرتبة تصاعدي .

1	3	6	8	9

Example :

```
# Input : Array A
# Output : Sorted array A

Algorithm: Selection_Sort(A)
for i ← 1 to n-1 do

    minj ← i
    minx ← A[i]

    for j ← i+1 to n do

        if A[j] < minx then
            minj ← j
            minx ← A[j]

    A[minj] ← A[i]
    A[i] ← minx
```

n

n * n

رح اكرر العملية من أول عنصر لـ لآخر عنصر n-1 بالـ array عرفت min j = index مكان أصغر عنصر والـ min x القيمة نفسها بفترض إنها أصغر عنصر .

if A[j] < min x then

إذا لقيت عنصر أصغر من الحالي min x بحدث min j = j مكانه وبحدث min x = A[j] قيمته .

A[min j] ← A[i] , A[i] ← min x

بعد ما اخلص البحث عن الأصغر ببدل الأصغر مع العنصر الحالي .

Big O notation (worst case) = n^2

Big Ω notation (best case) = n^2

Big Θ notation (average case) = n^2

Bubble Sort

بهاي الطريقة بقارن كل عنصر مع العنصر يلي جمبه وإذا كانوا مش بترتيب صح بعملهم swap العملية هاي بكررها أكثر من مرة لحد ما تطفو (bubble) العناصر الكبيرة لآخر الـ array وتصير مرتبة .

1	7	2	10	8
n=0	n=1	n=2	n=3	n=4

1	7	2	10	8

رح تضل نفسها لانه 1 اصغر من الـ 7 .

1	7	2	10	8

→

1	2	7	10	8

بعمل swap بين الـ 7 و الـ 2 .

1	2	7	10	8

→

1	2	7	10	8

رح تضل نفسها لانه 7 اصغر من الـ 10 .

1	2	7	10	8

→

1	2	7	8	10

وبكرر هاي الخطوات عالـ array من البداية كمان مرة .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Example :

```
# Input: Array A
# Output: Sorted array A

Algorithm: Bubble_Sort(A)

int flag = 1;

for i = 1 to n - 1 do
    for j = 1 to n - i do

        if A[j] > A[j + 1] then

            flag = 0;
            swap(A[j], A[j + 1])

    if (flag == 1)
        cout << "already sorted" << endl;
        break;
```

عرفت متغير برا الـ loop كلها اسمه flag قيمته 1 ، اذا كانت عندي الـ array مرتبة من البداية الشرط بالـ if مش رح يتحقق يعني قيمة الـ flag مش رح تصير صفر ولا رح يصير swap ويدخل عالشرط الثاني ويعمل break .

بس اذا كانت مش مرتبة بفحص كل عنصر مع يلي جمبه اذا اصغر منه او لا ، اذا تحقق الشرط فالـ array مش مرتبة بغير قيمة الـ flag و بعمل swap .

Big O notation (worst case) = n^2

Big Ω notation (best case) = n

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Insertion Sort

بهاي الطريقة كأني برتب بكروت الشدّة باخذ ورقة من كومة الأوراق يلي عالطاولة وببلش اختار وارتب فيهم اصغر او اكبر حسب الترتيب بس امسك ورقة عن الطاولة بقارنها بيلي انا ماسكتهم بايدي وبحطها بمكانها المناسب حسب الترتيب كل مرّة الورق بايدي بزيد وبكون مرتب نفس الفكرة بالـ Insertion Sort بعملها على الـ array .

2	7	1	4	3	9

بشوف اول index هو 2 ، فـ بحطه عجهة الـ sorted

2	7	1	4	3	9

sorted | unsorted

الـ index يلي بعد 7 ، هل الـ 7 اكبر من الـ 2 ؟ اه ، بحطها بعد الـ 2

2	7	1	4	3	9

sorted | unsorted

بعدها الـ 1 ، هل الـ اكبر من الـ 7 ؟ لا ببدلهم

2	1	7	4	3	9

sorted | unsorted

هل 1 اكبر من 2 ؟ لا ببدلهم

1	2	7	4	3	9

sorted | unsorted

الـ index يلي بعد 4 ، هل 4 اكبر من 7 ؟ لا ببدلهم

1	2	4	7	3	9

sorted | unsorted

هل 4 اكبر من 2 ؟ اه بخليها مكانها

الـ index يلي بعد 3 ، هل 3 اكبر من 7 ؟ لا ببدلهم

1	2	4	3	7	9

sorted ← | → unsorted

هل 3 اكبر من 4 ؟ لا ببدلهم

1	2	3	4	7	9

sorted ← | → unsorted

هل 3 اكبر من 2 ؟ اه بخليها مكانها

الـ index يلي بعد 9 ، هل 9 اكبر من 7 ؟ اه بخليها مكانها

وهيك صارت الـ array مرتبة .

1	2	3	4	7	9

Example :

```
# Input: Array T
# Output: Sorted array T

Algorithm: Insertion_Sort(T[1,…,n])

    for i ← 2 to n do
        x ← T[i];
        j ← i - 1;

        while x < T[j] and j > 0 do

            T[j+1] ← T[j];
            j ← j - 1;

        T[j+1] ← x;
```

n

n * n

ببلش من العنصر الثاني (i=2) لأنه العنصر الأول لحاله دايماً بعتبره مرتب كل مرة رح امسك عنصر جديد واحاول احطه بمكانه الصح بين العناصر يلي قبله .

```
for i ← 2 to n do                    // ببلش من العنصر الثاني
    x ← T[i]                         // بخزن العنصر الحالي
    j ← i - 1                        // بمسك العنصر يلي قبله
    while j > 0 and x < T[j]
        T[j+1] ← T[j]                // بزيح العنصر الكبير لـ اليمين
        j ← j - 1                    // وبرجع خطوة لورا
    T[j+1] ← x                       // بالمكان الصح x بحط
```

while x < T[j] and j > 0 do

رح اضل ارجع لورا (باتجاه اليسار) طول ما العنصر الحالي x أصغر من يلي قبله الشرط j > 0 عشان ما اطلع برا حدود الـ array واذا كانت مرتبة مش رح يتحقق الشرط ويدخل عالـ loop .

x = 6

j = 1

5	7	6
	j	i

x > A [i] 6 > 7 ? اه

A [j+1] = A [j]

A [2] = A [1]

j = j – 1 // 0 = 1 - 1

A [j+1] = x

A [0 + 1] = 6 // A [1] = 6

5	6	7

Big O notation (worst case) = n^2

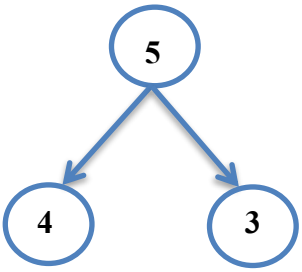
Big Ω notation (best case) = n

Heap Sort

العناصر مرتبة بطريقة بتشبه الشجرة والتفرعات يلي فيها طريقة مش خطية ، برتبها على شكل node وكل node يكون فيها تفرع وبيأشر على تفرع ، ببلش من الـ root تفرع لـ اليسار وبعدها تفرع لـ اليمين عشان تكون متوازنة الـ root بيأشر كحد اقصى على tow node وكل node إلها أب

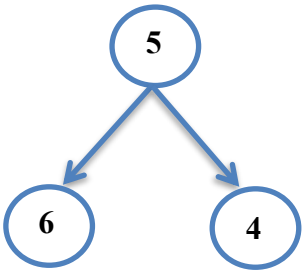
شروطها :

- Completely filled (متوازنة بالتفرعات) .
- max (كل الـ node يلي تحتها اصغر منها) .



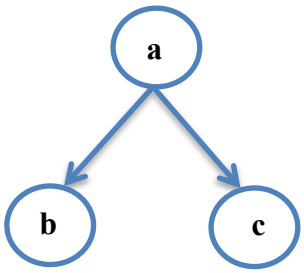
Heap

كل node اب اكبر من يلي تحته .



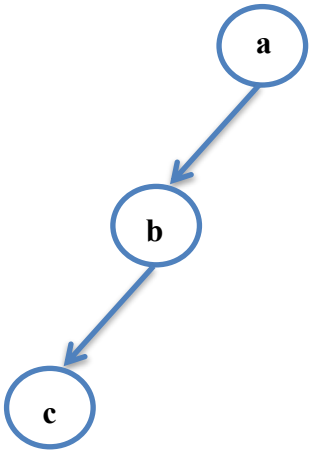
Not Heap

الـ 6 اكبر من الـ 5 .



Heap

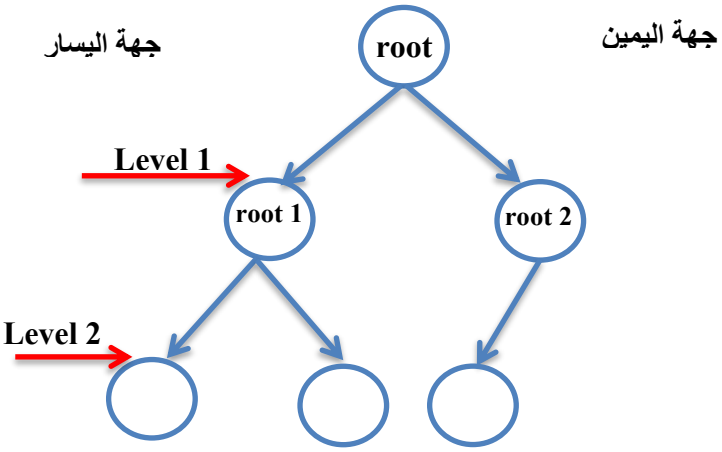
متوازنة بتفرعاتها completely filled .



Not Heap

لازم كل تفرع ابلش فيه من الـ root بعدها تفرع لـ اليسار
و بعدها تفرع لـ اليمين عشان تكون متوازنة
بس بالمثال هاد not completely filled .

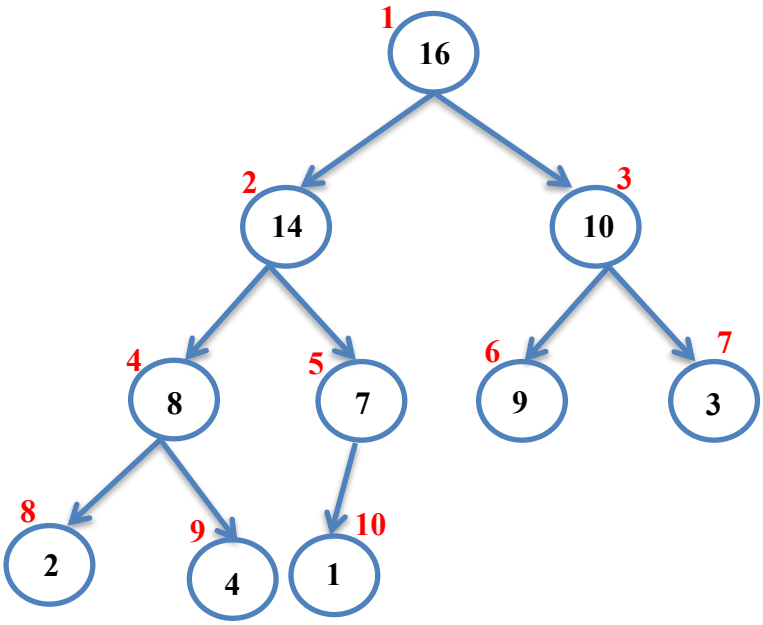
لو كان عندي 6 node كيف ارتبهم ؟



ببلش من الـ root بضيفه .

بعدها ببلش بالـ level 1 من جهة التفرع اليسار بضيف root 1 بـ level 2 بضيف تفرع لـ جهة اليسار
وبعدها لـ اليمين ، برجع لـ level 1 لجهة اليمين بضيف root 2 بـ level 1 بضيف تفرع لـ جهة اليسار
وبخلصو الـ node .

Example :



16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10
Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Index 6	Index 7	Index 8	Index 9

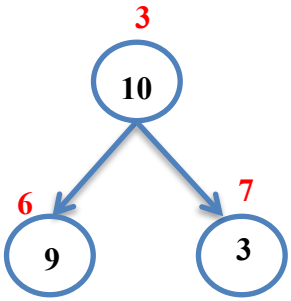
لو مثلاً جيت اخذت العنصر 14 ترتيبه بالـ array ؟ 2

الـ 14 هي الاب (parent) وأولادها (child) هم الـ 8 و 7

ترتيب الـ 8 بالـ array ؟ 4 وترتيب الـ 7 بالـ array ؟ 5

العلاقة بين ترتيب الـ 14 والابن يلي عجهة اليسار 8 الضعف يعني ترتيب الـ 14 = 2 وترتيب الـ 8 = 4 الابن يلي عجهة اليمين الـ 7 ترتيبه 1 + 4 = 5 .

لو مثلاً الـ child 3 يلي عاليمين ترتيبه 7 كيف اعرف ترتيب الاب ؟



- ترتيب الاب * 2 = ترتيب الابن عاليسار
- ترتيب الاب * 2 + 1 = ترتيب الابن عاليمين

ترتيب الاب = (ترتيب الابن عاليمين – 1) / 2

ترتيب الاب = 7 – 1 / 2 = 6 / 2 = 3

ترتيب الاب 3 صح وهو الـ 10 .

Types of Heap

Min Heap

كل الـ node يلي تحتها بتساويها او اكبر منها .

Where the value of the root node is less than or equal to either of its children .

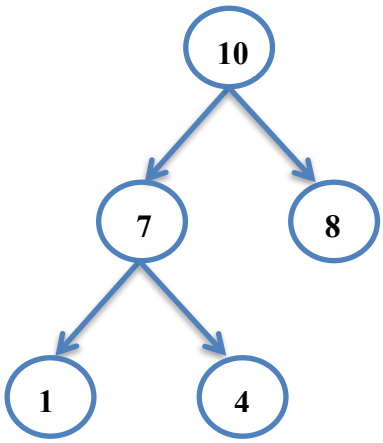
Max Heap

كل الـ node يلي تحتها بتساويها او اصغر منها .

Where the value of the root node is greater than or equal to either of its children .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Example :

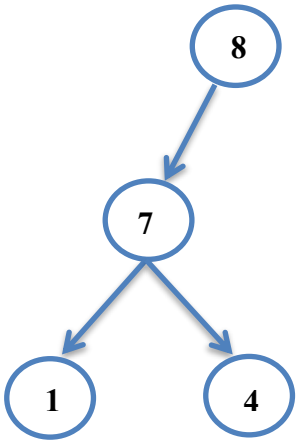


هاي max heap كل الـ node يلي تحت الـ 10 اصغر منه .

كيف اعملها heap sort ؟

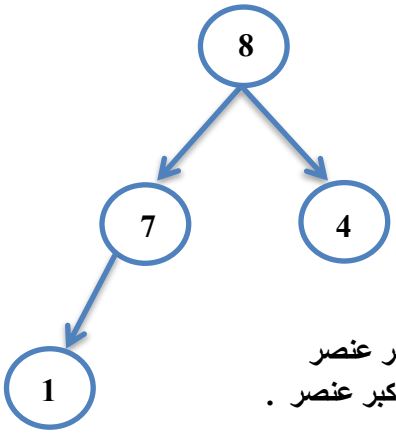
باخذ الـ 10 وبطبعها ، هيك طبعت اكبر عنصر ، وبحذفه من الـ tree
وبحط مكانه ثاني اكبر عنصر .

10



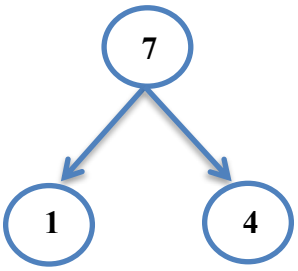
بس هيك بطلت completely filled

باخذ اي عنصر من أبناء الـ 7 وبحطو عاليمين بـ level 1 هو مش الحل الأمثل وبيغلّب بس عشان نفهم
ونوصل لـ الحل الصح .



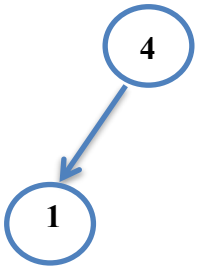
باخذ الـ 8 وبطبعها ، هيك طبعت ثاني اكبر عنصر
وبحذفه من الـ tree وبحط مكانه ثالث اكبر عنصر .

10	8



باخذ الـ 7 وبطبعها ، هيك طبعت ثالث اكبر عنصر وبحذفه من الـ tree وبحط مكانه رابع اكبر عنصر .

10	8	7



باخذ الـ 4 وبطبعها وبعدها بطبع الـ 1 يلي رح تضل .

10	8	7	4	1

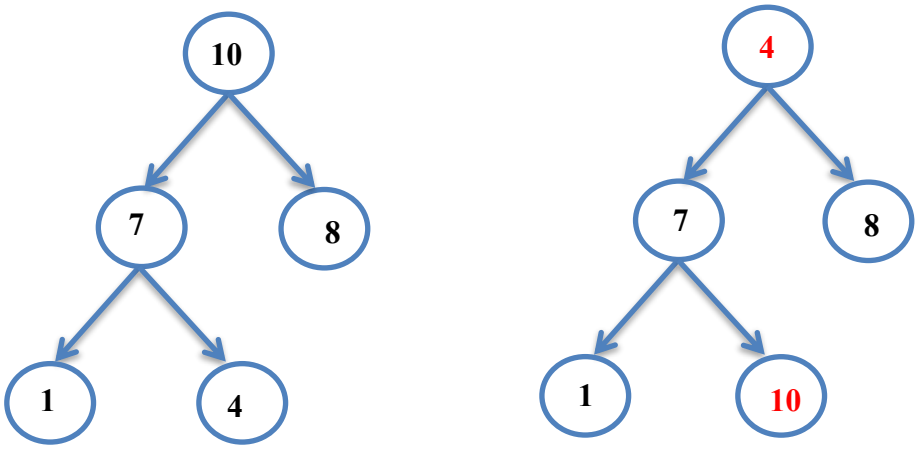
sorted

هي فكرة الـ max انه اطبع الـ root واحذفه من الـ tree واحط مكانه اكبر عنصر بضل واضل محافظة انه الـ tree تضل completely filled .

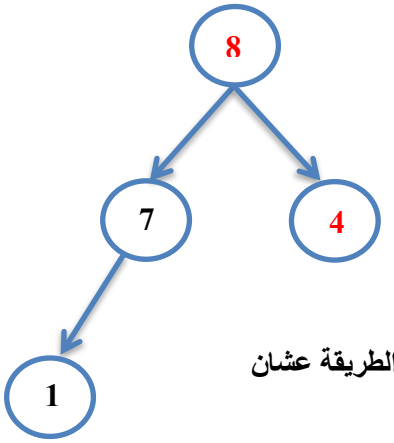
طريقة الحل الأمثل :

باخذ الـ 10 الـ root وبطبعه وببدل بينه وبين اخر عنصر ضفته عالـ tree .

10



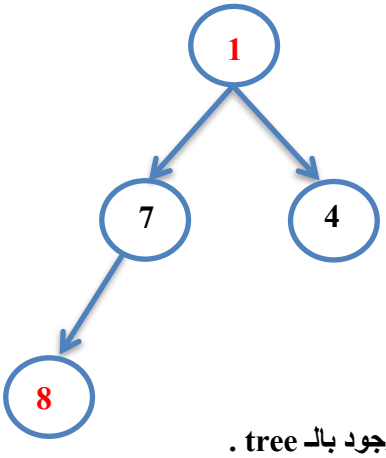
هسا بحذف الـ 10 ، وصارت completely filled بس مش max عشان اخليها max ببدلها مع اكبر عنصر موجود بالـ tree .

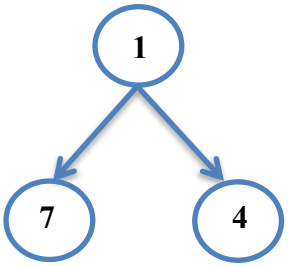


لو في node تحت الـ 4 بعمل نفس الطريقة عشان احافظ عالـ max

بكرر نفس الخطوة باخذ الـ 8 وبطبعها وببدل بينها وبين اخر عنصر بالـ tree .

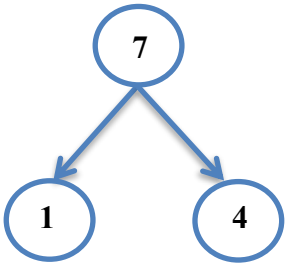
10	8



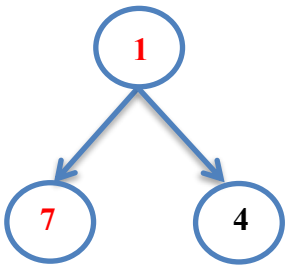


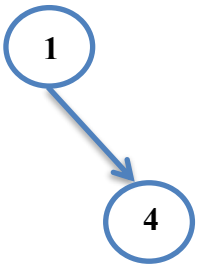
وبحذف الـ 8

وهسا عشان اخليها max ببدل الـ1 مع اكبر عنصر موجود بالـ tree .

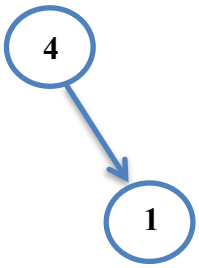


بطبع الـ root وبكمل نفس الخطوات .





10	8	7



ببدل بينهم عشان اخلي الـ tree تكون max

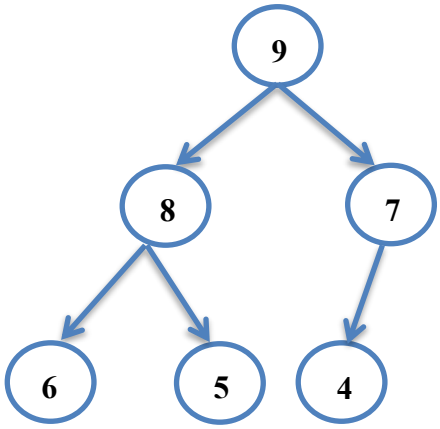
بطبع الـ 4 وبحذفها من الـ tree وبعدها بضل بس عندي الـ 1 بطبعه

10	8	7	4	1

sorted

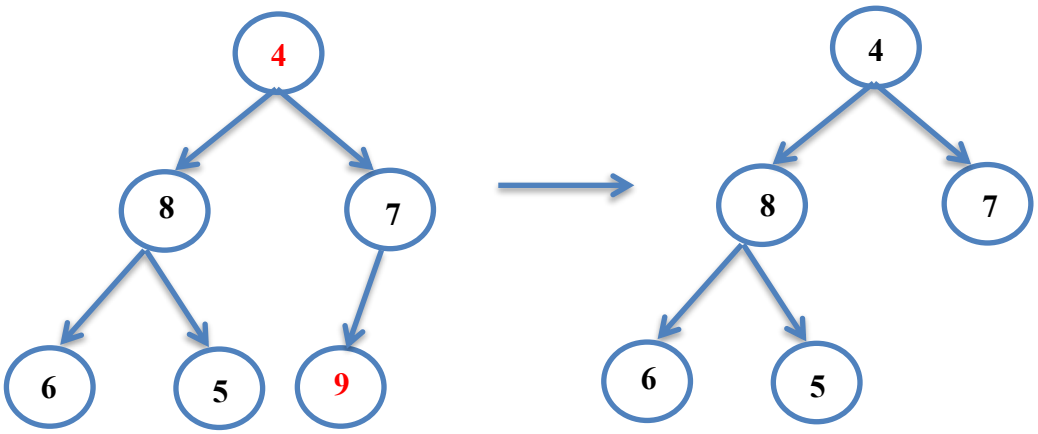
- Print max root
- Swap (last element , root)
- Delete last element
- Max heap

Example :



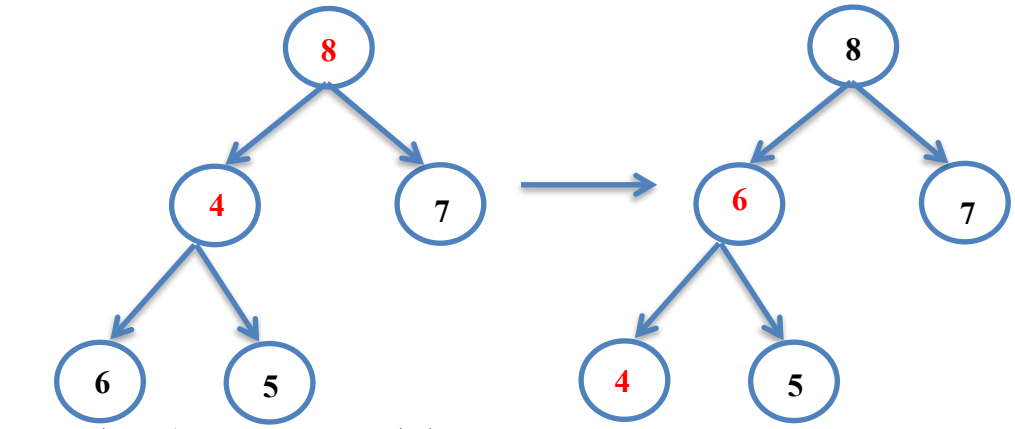
بطبع الـ 9 الـ root وببدله مع اخر عنصر ضفته بالـ tree .

9



بحذف الـ 9

ببدل الـ 4 مع اكبر عنصر عشان اخليها max

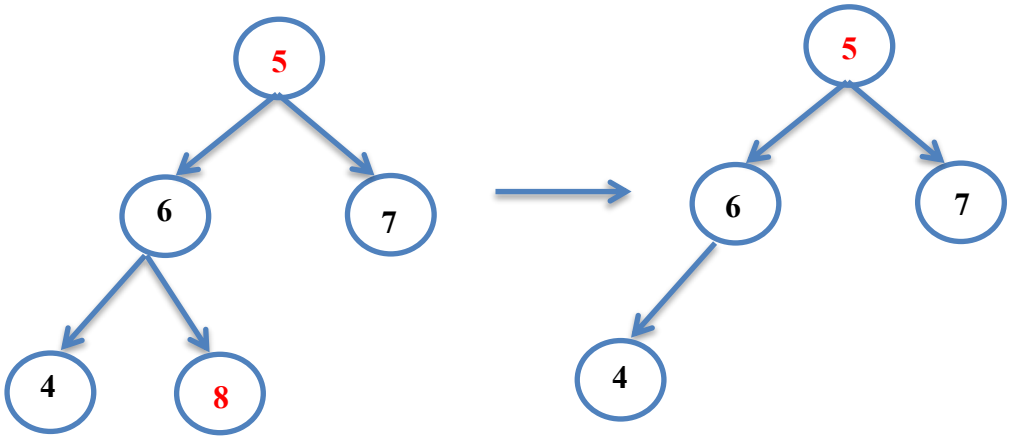


ببدل الـ 4 مع اكبر عنصر عشان اخليها max

هيك صارت max و completely filled

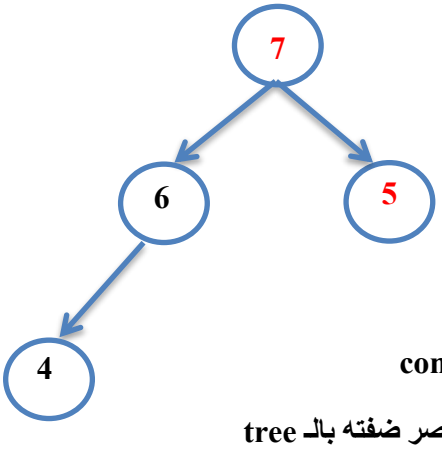
بطبع الـ 8 الـ root وببدله مع اخر عنصر ضفته بالـ tree

9	8



بحذف الـ 8

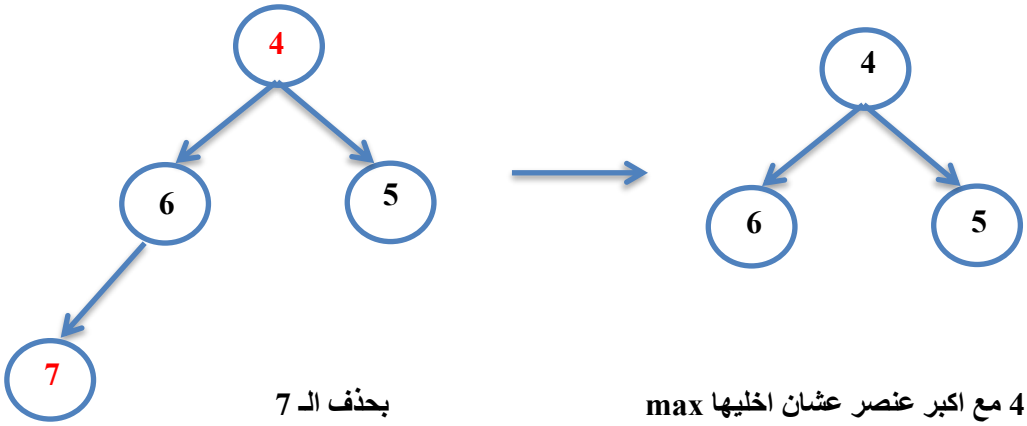
ببدل الـ 5 مع اكبر عنصر عشان اخليها max



هيك صارت max و completely filled

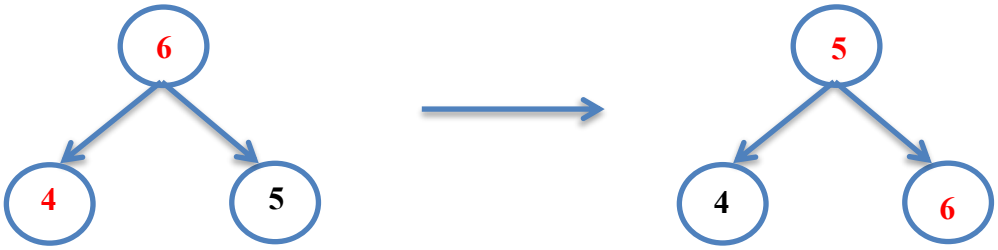
بطبع الـ 7 الـ root وببدله مع اخر عنصر ضفته بالـ tree

9	8	7



بحذف الـ 7

ببدل الـ 4 مع اكبر عنصر عشان اخليها max

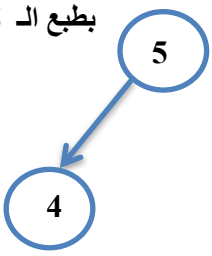


هيك صارت max و completely filled

بطبع الـ 6 الـ root وببدله مع اخر عنصر ضفته بالـ tree

9	8	7	6

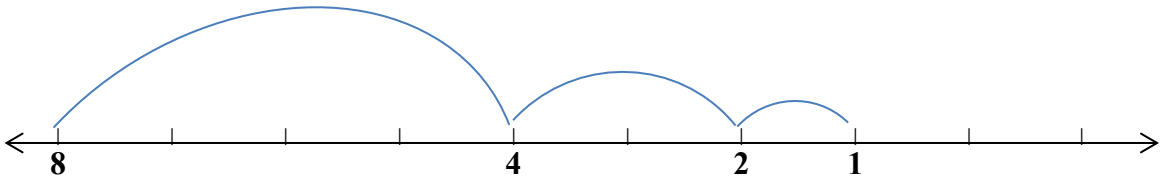
بطبع الـ 5 وبعدها 4



9	8	7	6	5	4

sorted

انا بكل مرة بكرر الخطوات ، بروح بفحص نُص الاعداد يلي ضلت وهاد بذكرنا بـ 2 =/ j او 2 =* j



الـ increment بقل بشكل مش متساوي ، يعني $\log_2 n$

في عملية الـ swap بـ max heap اخذت $\log_2 n$ ونفذتها على كل الـ node يعني n مرة نفذتها $n \log_2 n$

Big O notation (worst case) = $n \log_2 n$

Big Ω notation (best case) = $n \log_2 n$

Big Θ notation (average case) = $n \log_2 n$

	Selection Sort	Insertion Sort	Bubble Sort	Heap Sort
Big O notation (worst case)	n^2	n^2	n^2	$n \log_2 n$
Big Ω notation (best case)	n^2	n	n	$n \log_2 n$
Big Θ notation (average case)	n^2			$n \log_2 n$

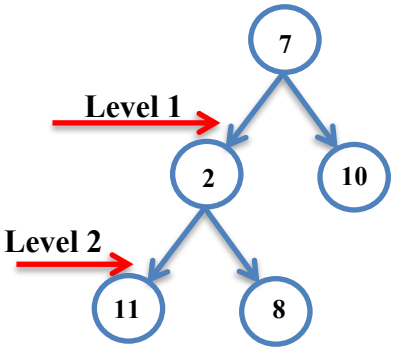
"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Build max heap

مثلاً انا عندي array ، كيف بخزنها عشكل tree ؟ ببلش من الـ root و بعدها من التفرع اليسار وبعدها التفرع اليمين بـ level 1 ، بعدها لـ level 2 بالتفرع اليسار وبعدها التفرع اليمين وع هالنمط .

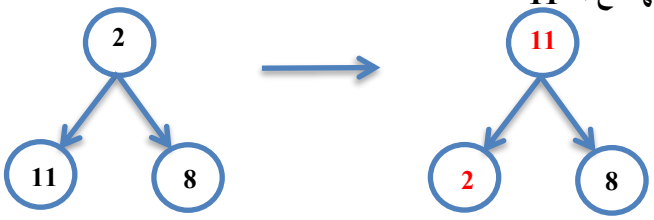
Example :

7	2	10	11	8

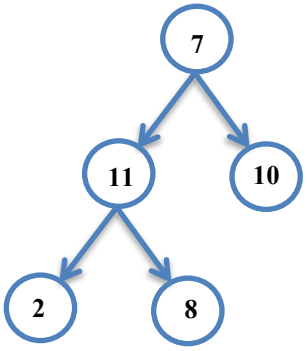


الـ tree حققت شرط انها completely filled بس الـ tree مش max heap .

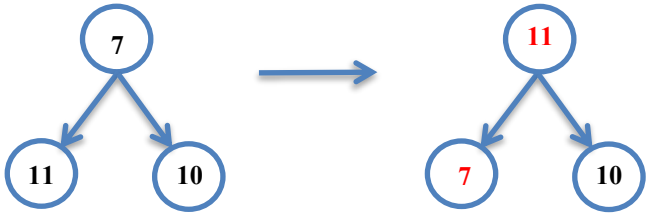
باخذ اخر level موجود من تحت ببلش دايماً ، هل الـ 2 اكبر من يلي تحتها ؟ لا ببدلها مع اكبر عدد من يلي تحتها (8 ، 11) ببدلها مع الـ 11



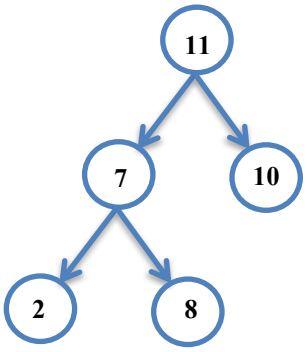
رح يصير هيك شكلها



هسا باخذ الـ level يلي فوق

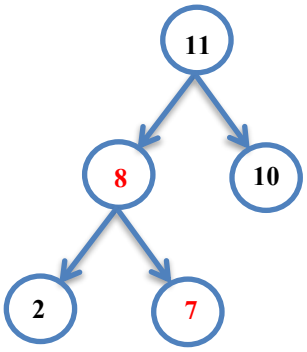


رح يصير هيك شكلها



الاب الـ 11 اكبر من الأبناء 10 و 7 ؟ اه

الاب الـ 7 اكبر من الأبناء 8 و 2 ؟ لا ببدل بين الـ 7 والـ 8

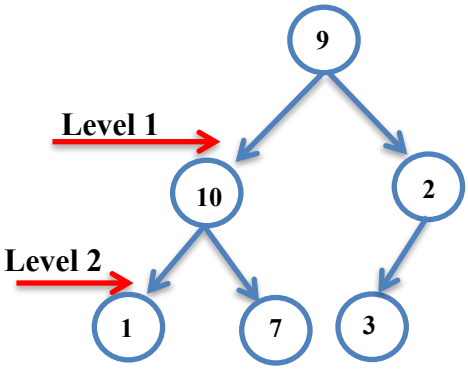


هيك صارت max heap

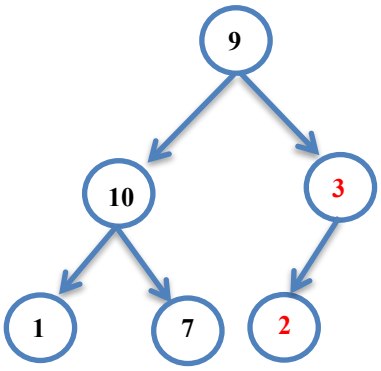
وهيك بقدر اطبق الـ heap sort .

Example :

9	10	2	1	7	3

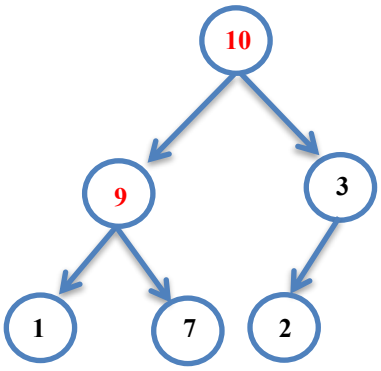


باخذ اخر level موجود من تحت ببلش من اليسار ، هل الـ 10 اكبر من يلي تحتها ؟ اه ، هيك هاد الـ level تمام ، بعدها الاب 2 والابن 3 الـ 2 اكبر من الـ 3 ؟ لا ببدلهم



باخذ الـ level الأول

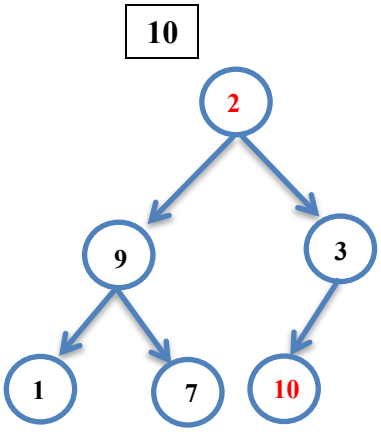
الاب 9 اكبر من الأبناء 3 و 10 ؟ لا ببدله مع اكبر عدد تحته 10



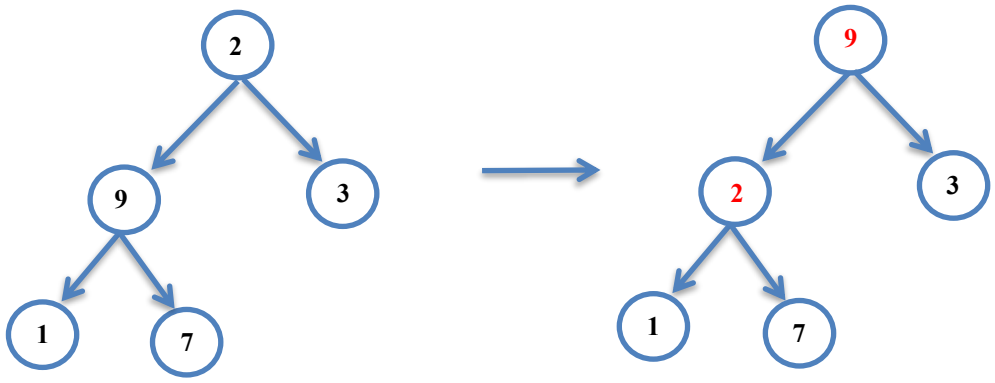
هيك صارت max heap

لو بدي اكمل واطبق الـ heap sort

الـ 10 الـ root بطبعها وببدلها مع اخر اشي ضفته عالـ tree

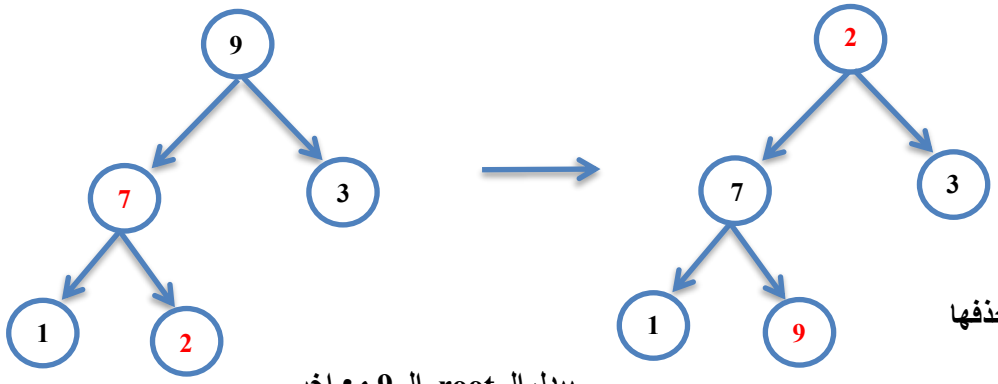


هسا بحذف الـ 10 ، وبحاول اخلي الـ tree تضل max heap و completely filled .



ببدل الـ 2 مع اكبر عنصر عشان اخليها max

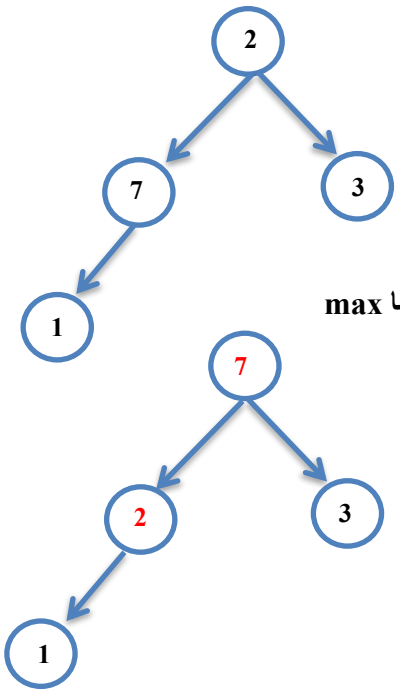
وبرجع كمان مرة ببدل الـ 2 مع اكبر عنصر تحته الـ 7 .



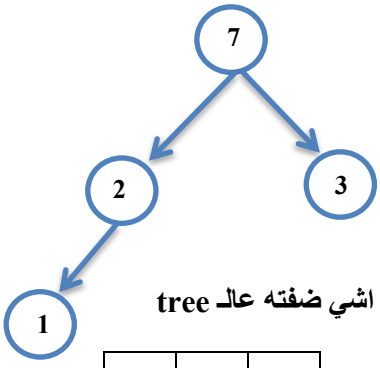
بطبع الـ 9 وبحذفها

ببدل الـ root الـ 9 مع اخر
اشي ضفته بالـ tree

10	9

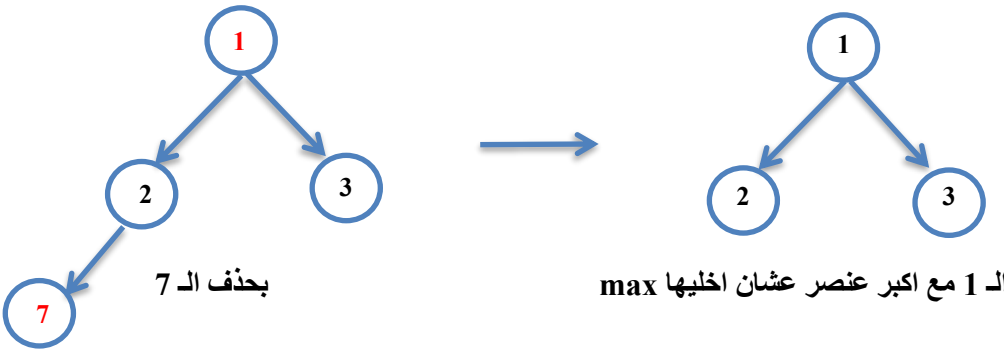


ببدل الـ 2 مع اكبر عنصر عشان اخليها max



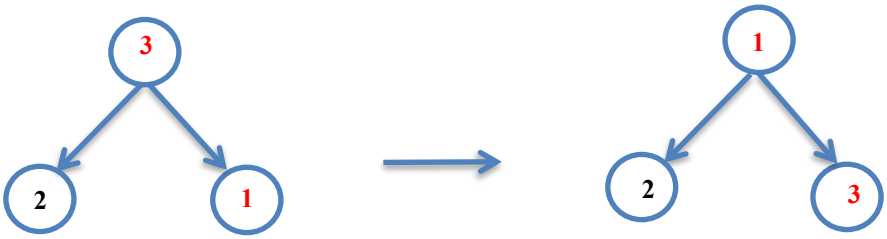
الـ 7 الـ root بطبعها وببدلها مع اخر اشي ضفته عالـ tree

10	9	7



بحذف الـ 7

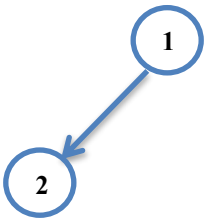
ببدل الـ 1 مع اكبر عنصر عشان اخليها max



بحذف الـ 3

بطبع الـ root الـ 3 وببدل بينه وبين اخر اشي ضفته عالـtree

10	9	7	3



ببدل بين الـ 2 والـ 1

بطبع الـ 2 بالاول وبعدها الـ1

10	9	7	3	2	1

Sorted

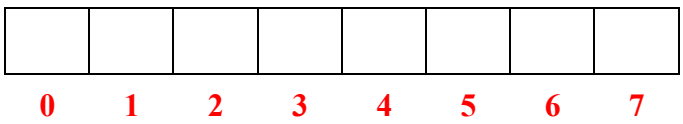
```
BUILD Max_HEAP(A)                    → ميثود جاهزة لـ البناء
    for i ← length[A] downto 2
        do exchange A[1] ↔ A[i]
        heap_size[A] ← heap_size[A] - 1
        MAX_HEAPIFY(A,1,n)           → ميثود
```

الكود بشتغل على نقل أكبر عنصر لمكانه الصح كل مرة بنهاية الـ array بصغّر حجم الـ heap تدريجياً و بستخدم MAX_HEAPIFY عشان يرجّع الترتيب ويضل الـ heap صح .

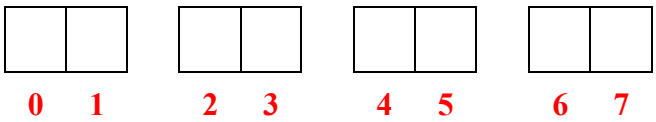
Start of Divide and Conquer

هاي فكرة عامة بالـ algorithm ، بتعتمد على انه اقسم المشكلة الكبيرة لمشاكل أصغر وأسهل حلّهم ، وبعدها ادمج الحلول مع بعض عشان اوصل لحل المشكلة الأصلية .

بكون عندي array مثلاً بدي اوجد العنصر x بالـ array



بقسم هاي الـ array



على فرض عندي method اسمه find x رح يستقبل كل قسم بنفس الوقت ، واذا لقي x بأحد الاقسام يرجع true ، واذا لا يرجع false ، يعني أنا لو فحصت الـ array كاملة رح ياخذ وقت طويل بس لما يفحص 4 اقسام بنفس الوقت ، رح يكون الوقت أقل بكثير يعني وقت تنفيذ جزء واحد مش array كاملة .

مثلاً عندي شخص واحد واعطيته الـ array يدور على x رح يأخذ وقت عشان يلاقيها لانه رح يدور بالـ array كاملة ، بس لو قسمت الـ array لاقسام واعطيت كل قسم لـ شخص 4 اشخاص وحكيتلهم دورو ع x رح نلاقيها بوقت اقل (مبدأ فَرّق تَسد) .

أغلب الـ Divide and Conquer مبنية على الـ Recursion ، لأنه أنا كل مرة بقسم المشكلة لمشاكل أصغر ، وهيك طبيعي ارجع انادي نفس الدالة Recursion على الجزء الأصغر .

Recursion

على فرض وقفت بطابور وانا كنت اخر شخص بـ هاد الطابور ، وبدي اعرف كم حدا قدامي وسألت يلي قدامي كم حدا قدامنا ؟ وهو برضو ما بعرف فـ بضل كل شخص يسأل يلي قدامو لنوصل أول حدا بالطابور ويحكي انو فش حدا قدامه ، وهيك بنصير نرجع رجوع وبعرف كم حدا قدامي .

الاستدعاء الذاتي (Recursion) : مثلاً لو عندي problem بضل اقسمها لـ problem اصغر لأوصل لحل

Base case : الجزء يلي بوصلّه بالحل وبوقف عنده وبرجع رجوع (لما وصلت اول حدا بالطابور ما في حدا قدامي) .

Solution : دمج الـ solution يلي بعد ويلي قبل .

Recursion

- **A programming technique that breaks down a complex problem into smaller , manageable pieces .**
- **Recursive solutions solve a problem by applying the same algorithm to each piece and then combining the results .**

اشهر مثال على الـ Recursion هو الـ factorial

$$5! = 5 * 4!$$

$$4! = 4 * 3!$$

$$3! = 3 * 2!$$

$$2! = 2 * 1!$$

$$1! = 1$$

لو عندي method لـ حساب الـ factorial

120 = 5 * 24

fact (5) = 5 * fact (4)

24 = 4 * 6

fact (4) = 4 * fact (3)

6 = 3 * 2

fact (3) = 3 * fact (2)

2 = 2 * 1

fact (2) = 2 * fact (1)

1 = 1 * 1

fact (1) = 1

قسمت المشكلة وجمعت الحلول

fact (1) = 1

```
int Factorial1() {
    int n = 1;
    if (n == 0 || n == 1)
        return 1;
    else
        return 1 * Factorial1();
}
```

fact (2) = 2 * fact (1)

```
int Factorial2() {
    int n = 2;
    if (n == 0 || n == 1)
        return 1;
    else
        return 2 * Factorial1();
}
```

fact (3) = 3 * fact (2)

```
int Factorial3() {
    int n = 3;
    if (n == 0 || n == 1)
        return 1;
    else
        return 3 * Factorial2();
}
```

fact (4) = 4 * fact (3)

```
int Factorial4() {
    int n = 4;
    if (n == 0 || n == 1)
        return 1;
    else
        return 4 * Factorial3();
}
```

fact (5) = 5 * fact (4)

```
int Factorial5() {
    int n = 5;
    if (n == 0 || n == 1)
        return 1;
    else
        return 5 * Factorial4();
}
```

```java
public static int factorial(int n) {
    if (n == 0 || n == 1) { // Base case
        return 1;
    } else {
        return n * factorial(n - 1); // Recursive call
    }
}
```

كأني بطابور رح اسأل يلي قدامي كم حدا ضل ، ويلي قدامي رح يسأل يلي قدامه لحتى نوصل لأول حدا بالطابور بس هالأسئلة بعرف كم مرة رح تتكرر ؟؟ لا ما بعرف

يعني الـ factorial (n-1) ما بعرف كم مرة رح تتكرر ويضل الـ fun يستدعي يلي قبله .

كيف اعرف كم الـ Running Time لـ كود الـ Recursion ؟

لو كانت n = 1 او 0 ما في داعي ادخل لجملة else لانه هاد الـ Base case لانه أول الطابور وبعرف مين قدامي يعني مرة وحدة رح يتنفذ ، وبنفذ بس return 1 .

لو اكبر من 1 ، رح يدخل عالـ Recursive call ترتيبي بالطابور ما بعرف كم ووين واقفة وكم حدا قدامي رح ينفذ امر الـ n يعني مرة وحدة و الامر factorial (n – 1) ما بعرف كم مرة بس رح تتنفذ بمقدار الـ Running Time - 1 .

$$\text{Running Time} = \begin{cases} (n = 0 \,||\, n = 1)\ ,\ RT = 1 \\ n > 1\ ,\ RT = 1 + RT(n-1) \end{cases}$$

بدلالة F(n)

$$F(n) = \begin{cases} (n = 0 \,||\, n = 1)\ ,\ F(n) = 1 \\ n > 1\ ,\ F(n) = 1 + F(n-1) \end{cases}$$

Calculate Running Time Using Substitution Method

Example :

كل استدعاء لـ F(n) بيرجع (1 + الاستدعاء اللي قبله)
ببلش من الـ Base case أصغر قيمة لـ n من F (1) = 1

$$F(n) = 1 + F(n-1)$$

بعوض n = n - 1

$$\boxed{F(n-1) = 1 + F((n-1)-1) = 1 + F(n-2)}$$

$$F(n) = 1 + 1 + F(n-2)$$

$$F(n) = 2 + F(n-2)$$

$$\boxed{F(n-2) = 1 + F((n-2)-1) = 1 + F(n-3)}$$

$$F(n) = 1 + 1 + F(n-3)$$

$$F(n) = 3 + F(n-3)$$

رح أوصل لمرحلة تكون قيمة الـ 3 اكتر ، 4 ، 5 ..... لـ k

$$F(n) = k + F(n-k)$$

رح اضل امشي بالاستدعاء لحتى يكون (n – k) = 1

فلو بدي اوجد قيمة k من المعادلة (n – k) = 1 ، رح تكون k = n - 1

بدي اعوض قيمة k // $F(n) = k + F(n-k)$

$$F(n) = (n-1) + F(n-(n-1))$$

$$F(n) = (n-1) + F(n-n+1)$$

$$F(n) = (n-1) + F(1)$$

$$F(n) = (n-1) + 1$$

$$F(n) = n - 1 + 1$$

$$F(n) = n$$

الـ F (n) هي رح اعتبرها الـ Running Time

Big O notation = O (n)

Example :

```
int fun(int n) {

    if (n == 1) {
        return 1;
    }

    for (int i = 0; i < n; i++) { }

    return n + fun(n - 1);

}
```

اذا كانت n = 1 رح يتنفذ الكود بس مرّة

اذا كانت n > 1 رح يتنفذ الكود بالـ loop (n) مرّة

اذا كانت n > 1 رح يتنفذ F (n - 1) مرّة

$$F(n) = \begin{cases} (n = 1)\ ,\ F(n) = 1 \\ n > 1\ ,\ F(n) = n + F(n-1) \end{cases}$$

The Base case

F (1) = 1

$$F(n) = n + F(n-1)$$

بعوض n = n - 1

$$F(n) = n + F(n-1)$$

$$\boxed{F(n-1) = n-1 + F((n-1)-1) = n-1 + F(n-2)}$$

$$F(n) = n + n - 1 + F(n-2)$$

$$F(n) = 2n - 1 + F(n-2)$$

$$\boxed{F(n-2) = (n-1)-1 + F((n-1)-1) = n-2 + F(n-3)}$$

$$F(n) = 2n - 1 + n - 2 + F(n-3)$$

$$F(n) = 3n - 3 + F(n-3)$$

$$\boxed{F(n-3) = (n-1)-2 + F((n-1)-2) = n-3 + F(n-4)}$$

$$F(n) = 3n - 3 + n - 3 + F(n-4)$$

$$F(n) = 4n - 6 + F(n-4)$$

$$\boxed{F(n-4) = (n-1) - 3 + F((n-1)-3) = n - 4 + F(n-5)}$$

$$F(n) = 4n - 6 + n - 4 + F(n-5)$$

$$F(n) = 5n - 10 + F(n-5)$$

$$\boxed{F(n-5) = (n-1) - 4 + F((n-1)-4) = n - 5 + F(n-6)}$$

$$F(n) = 5n - 10 + n - 5 + F(n-6)$$

$$F(n) = 6n - 15 + F(n-6)$$

متساويات الـ 6 بقدر احكي عنهم k ، و شو علاقة الـ 15 بـ الـ 6 ؟

مجموع $(1 + 2 + 3 + 4 + 5) = 15$ ، المجموع من 1 لـ $6 - 1 = 15$

$$15 = \frac{6(6-1)}{2}$$

$$\sum_{1}^{k-1} i = \frac{k(k-1)}{2}$$

$$F(n) = k\,n - \sum_{1}^{k-1} i + F(n-k)$$

رح اضل امشي بالاستدعاء لحتى يكون $(n-k) = 1$ ، من المعادلة $(n-k) = 1$ رح تكون $k = n - 1$

$$F(n) = (n-1)\,n - \sum_{1}^{(n-1)-1} i + F(n-(n-1))$$

$$F(n) = n^2 - n - \sum_{1}^{n-2} i + F(1)$$

الـ $F(n)$ هي رح اعتبرها الـ Running Time

باخذ اكبر power + بدون constant

Big O notation = $O(n^2)$

Example :

$$T(n) = \begin{cases} (n = 1)\ ,\ T(n) = 1 \\ n > 1\ ,T(n) = 1 + T(n/2) \end{cases}$$

The Base case

$F(1) = 1$

$$F(n) = 1 + F(n/2)$$

بعوض $n = n - 1$

$$F(n-1) = 1 + F(n-1/2) = 1 + F(n/2)$$

$$\boxed{F(n/2) = 1 + F(n/2/2) = 1 + F(n/4)}$$

$$F(n) = 1 + 1 + F(n/4)$$

$$F(n) = 2 + F(n/4)$$

$$\boxed{F(n/4) = 1 + F(n/4/2) = 1 + F(n/8)}$$

$$F(n) = 2 + 1 + F(n/8)$$

$$F(n) = 3 + F(n/8)$$

$$\boxed{F(n/8) = 1 + F(n/8/2) = 1 + F(n/16)}$$

$$F(n) = 3 + 1 + F(n/16)$$

$$F(n) = 4 + F(n/16)$$

$$\boxed{F(n/16) = 1 + F(n/8/2) = 1 + F(n/32)}$$

$$F(n) = 4 + 1 + F(n/32)$$

$$F(n) = 5 + F(n/32)$$

$$F(n) = 5 + F(n / 32)$$

في علاقة بين الـ 32 و الـ 5 ؟

$$32 = 2^5$$

$$F(n) = 5 + F(n / 2^5)$$

بعتبر الـ 5 هي k

$$F(n) = k + F(n / 2^k)$$

رح اضل امشي بالاستدعاء لحتى يكون $1 = (n / 2^k)$

$$1 = n / 2^k$$

ضرب تبادلي

$$n = 2^k$$

$$\log_2(n) = \log_2(2^k)$$

$$\log_2(n) = k \log_2(2)$$

$$\log_2(n) = k * 1$$

$$\log_2(n) = k$$

$$F(n) = \log_2(n) + F(n / 2^k)$$

الـ $F(n)$ هي رح اعتبرها الـ Running Time

باخذ اكبر power + بدون constant

Big O notation = $O(\log_2(n))$

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Calculate Running Time Using master theorem

هاي الطريقة الاسهل بس مش دايماً بتزبط ، بوجد فيها الـ Big O من النَظرة بس فيها شرط انه

$$T(n) = a\,T(n/b) + F(n)$$

لو بدي اطبق الطريقة عالمثال يلي حلّيته قبل .

$$T(n) = a\,T(n/b) + F(n)$$

$$T(n) = T(n/2) + 1$$

$$a = 1 , b = 2 , F(n) = 1$$

بدي اعمل مقارنة بين

$$n^{\log_b a} \quad ? \quad F(n)$$

الأكبر بينهم بكون هو الـ Big O = (الأكبر) O

في حالة تساو ؟ الـ Big O = (وحدة منهم *) O (log(n)

$$T(n) = T(n/2) + 1$$

$$a = 1 , b = 2 , F(n) = 1$$

$$n^{\log_b a} \quad ? \quad F(n)$$

$$n^{\log_2 1} \quad ? \quad 1$$

0 =

$$n^{0} \quad ? \quad 1$$

$$1 \quad ? \quad 1$$

متساويات يعني Big O = وحدة منهم * (log (n

$$O(\log(n) * 1)$$

$$O(\log(n))$$

Example :

$$T(n) = 2T(n/2) + n$$

$a = 2$, $b = 2$, $F(n) = n$

$$n^{\log_b a} \quad ? \quad F(n)$$

$$n^{\log_2 2} \quad ? \quad n$$

$1 =$

$$n^{1} \quad ? \quad n$$

$$n \quad ? \quad n$$

متساويات يعني Big O = وحدة منهم * log (n)

$$O(\log(n) * n)$$

$$O(n \log(n))$$

Example :

$$T(n) = 2T(n/2) + n^2$$

$a = 2$, $b = 2$, $F(n) = n^2$

$$n^{\log_b a} \quad ? \quad F(n)$$

$$n^{\log_2 2} \quad ? \quad n^2$$

$1 =$

$$n^{1} \quad ? \quad n^2$$

$$n \quad ? \quad n^2$$

الـ F (n) اكبر Big O = O (F (n))

$$O(n^2)$$

Example :

$$T(n) = 9\,T(n/3) + n$$

$a = 9$, $b = 3$, $F(n) = n$

$$n^{\log_b a} \quad ? \quad F(n)$$

$$n^{\log_3 9} \quad ? \quad n$$

$\log_3 3^2 = 2 \log_3 3$

$2 \log_3 3 = 2 * 1 = 2$

$$n^2 \quad ? \quad n$$

$$n^2 \quad ? \quad n$$

Big O = O (الأكبر)

$O(n^2)$

Example :

$$T(n) = 7\,T(n/2) + n^3$$

$a = 7$, $b = 2$, $F(n) = n^3$

$$n^{\log_b a} \quad ? \quad F(n)$$

$$n^{\log_2 7} \quad ? \quad n^3$$

8 ≈ بدي اقربها 7

$\log_2 2^3 = 3 \log_2 2$

$3 \log_2 2 = 3 * 1 = 3$

$$n^{3} \quad ? \quad n^{3}$$

رح تكون 2.99 لانه قربتها
يعني الأكبر F (n)

الـ F (n) اكبر Big O = O (F (n))

$$O(n^3)$$

Example :

$$T(n) = 7T(n/2) + n^2 + n$$

$a = 7$, $b = 2$, $F(n) = n^2 + n$

قيمة F (n) ممكن تكون حد اكثر من term بشرط انه بدلالة n مش قيمة .

Example :

$$T(n) = 2T(n) + n$$

$a = 7$, $b = 1$, $F(n) = n^2 + n$

ما بقدر اطبق عليها طريقة الـ master قيمة b رح تكون بتساوي 1
$Log_1(2)$ غير معرف ، مستحيل 1 قوة أي عدد = 2 .

Example :

$$T(n) = T(n/2) + T(n/4) + n^2$$

ما بقدر اطبق عليها طريقة الـ master لانه ما بقدر احدد قيمة b .

لو كانت قيمة b متساوية ؟

$$T(n) = T(n/2) + T(n/2) + n^2$$

$$T(n) = 2T(n/2) + n^2$$

بقدر اطبق عليها طريقة الـ master قيمة b رح تكون بتساوي 2 و قيمة a بتساوي 2 .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Recursion Tree

$$T(n) = 2T(n/2) + n$$

دايماً الشكل العام بالمعادلة رح يضل نفسه ويتغير يلي جوا T قيمة الاستدعاء ، وكل لفة بتختلف قيمة n
ودايماً أي معادلة عندي هي قسمين ، قسم الـ Recursion ، وقسم الـ non - Recursion .

$$T(n) = 2\boxed{T(n/2)} + \boxed{n}$$

قسم الـ Recursion قسم الـ non - Recursion

$$T(n) = 2T(n/2) + n$$

$$\boxed{T(n/2) = 2T((n/2)/2) + n/2 = 2T(n/4) + n/2}$$

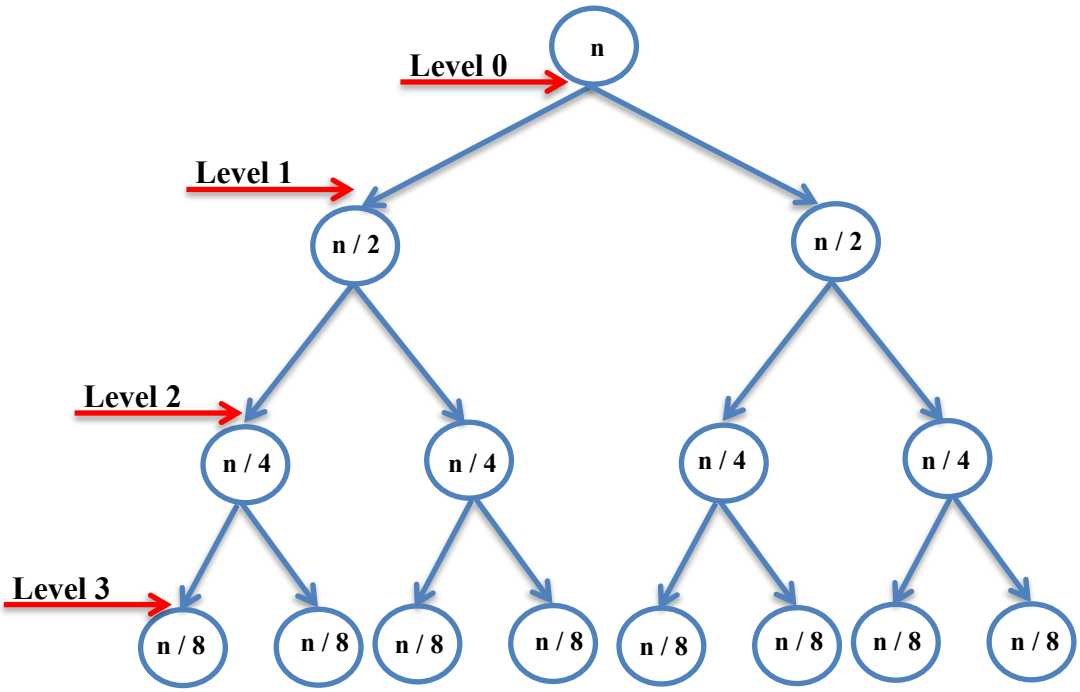
$$T(n/2) = 2T(n/4) + n/2$$

$$T(n/4) = 2T(n/8) + n/4$$

بكون عندي T (n) بعدها بستدعي T (n / 2) وبعدها T (n / 4) وهكذا .....

قسم الـ non - Recursion ، n هاد بعتبره الـ node (الـ root الأب n) .

قسم الـ Recursion ، هاد بعتبره الـ level 1 ، 2 T (n / 2) (2 node أبناء للـ n) .



رح اضل مكملة لحد مُعين .

لنفرض n = 32 بقسم على 2 بكل level

32

16

8

4

2

1

هيك عندي level 5 ببلش اعدهم من level 0 .

العلاقة بين 32 و 5 ؟ $2^5 = 32$

$$\log_2 32 = 5$$

عدد المستويات

مقدار n = 32

كل مرّة بقسم على 2 كل node رح ينبثق منها 2 node .

$$T(n) = 2T(n/2) + n$$

$a = 2 , b = 2 , T(n) = n$

$$\log_b n = \text{number of level}$$

عدد الاستدعاءات n لكل level (عدد الـ leaf او العناصر الموجودة بالـ level) .

$$2^{\log_2 32}$$

مقدار n = 32

كل مرّة بقسم على 2 كل node رح ينبثق منها 2 node .

$$a^{\log_b n} = \text{number of leaf on level}$$

عدد الـ Levels الارتفاع : الارتفاع height لـ الـ tree هو عدد الحواف edges من الجذر root إلى أبعد ورقة leaf باخذ أطول مسار من root لـ الـ leaf (دايماً أطول مسار) وبحسب عدد الـ edges يعني التفرعات يلي جوّا المسار (→) .

حجم العقدة عند المستوىi=bin

بالـ level 2 مقدار الـ leaf = n / 4

عند المستوى 2 = $n / 2^2$ = n / 4

كل level مجموع الـ node فيه = n

Level 0 = n

Level 1 = (n / 2) + (n / 2) = n

Level 2 = (n / 4) + (n / 4) + (n / 4) + (n / 4) = n

رح اضل امشي بالـ level وجمع عناصرهم = n ، الا اخر level = T (1) ما بقدر احكي عنه = n لانه اله قيمة ثابتة .

- عند كل level مجموع العمليات = n
- بضلني اجمعهم من الـ Level 0 لـ الـ level الأخير
- الـ level الأخير هو T (1) وقيمته ثابتة

$$T(n) = \boxed{\sum_{i=0}^{h-1} n} + \boxed{T(1) * (\text{عدد الـ } T(1) \text{ في الـ level الأخير})}$$

مجموع العمليات عند كل level من اول level = 0 لـ h -1 الارتفاع ، يعني لـ الـ level قبل الأخير .

قيمة الـ level الأخير T (n) * عدد الـ T (n) بالـ level

كنت اوجد عدد الـ leaf بكل level يعني عدد الاستدعاءات من العلاقة $a^{\log_b n}$

$$T(n) = \sum_{i=0}^{h-1} n + T(1) * 2^{\log_2 n}$$

$$T(n) = \sum_{i=0}^{h-1} n + T(1) * 2^{\boxed{\log_2 n}}$$

$$n^{\log_2 2} = n$$

$$T(n) = \sum_{i=0}^{h-1} n + T(1) * n$$

$$T(n) = \sum_{i=0}^{h-1} n + 1 * n$$

$$T(n) = \sum_{i=0}^{h-1} n + n$$

h : number of level الارتفاع يعني عدد المستويات

number of level = $\log_b n = \log_2 n$

$$T(n) = \sum_{i=0}^{Log2\, n} n + n$$

$\sum_{i=0}^{Log2\, n} n$ يعني رح اجمع الـ n بعدد الـ h (رح اجمع n بمقدار h مرّة) .

يعني $\log_2 n * n = h * n$

$$T(n) = n * \log_2 n + n$$

الـ T (n) هي رح اعتبرها الـ Running Time

باخذ اكبر power + بدون constant

Big O notation = O (n log n)

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Example :

$$T(n) = T(n/3) + T(2n/3) + n$$

$a = 1$, $b = 3$, $b = 2/3$, $T(n) = n$

الـ root هي $T(n) = n$ بتنقسم إلى 2 node

الأول $T(n/3)$

الثاني $T(2n/3)$

كل node في الـ tree بتمثل استدعاء تابع (recursive call) .

$$T(n) = T(n/3) + T(2n/3) + n$$

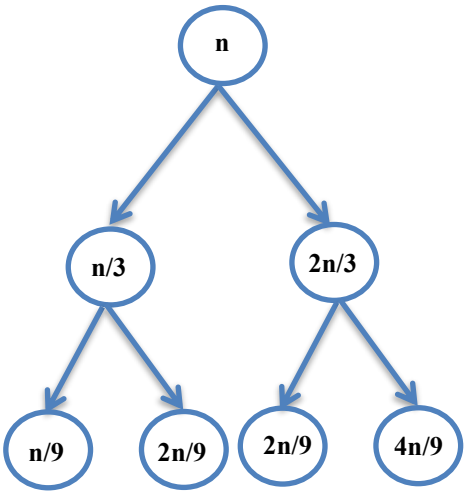
$$T(n/3) = T((n/3)/3) + T(2(n/3)/3) + n/3 =$$

$$= T(n/9) + T(2n/9) + n/3$$

$$T(n) = T(n/3) + T(2n/3) + n$$

$$T(2n/3) = T((2n/3)/3) + T(2(2n/3)/3) + 2n/3 =$$

$$= T(2n/9) + T(4n/9) + 2n/3$$



بضل مكملة ...

Level 0 = n

Level 1 = (n / 3) + (2n / 3) = 3n / 3 = n

Level 2 = (n / 9) + (2n / 9) + (2n / 9) + (4n / 9) = 9n / 9 = n

الارتفاع رح يكون شوية فيه اختلاف بين جهة اليمين واليسار ، بس رح احسب لعند أطول leaf اكبر b .

$$\text{Log}_b n = \text{number of level}$$

number of level = $\text{Log}_b n$ = $\text{Log}_{2/3} n$

$$a^{\text{Log}_b n} = \text{number of leaf on level}$$

number of leaf on last level = $a^{\text{Log}_b n} = 3^{\text{Log}_{2/3} n}$

$$T(n) = \sum_{i=0}^{h-1} n + T(1) * (\text{number of leaf on level})$$

$$T(n) = \sum_{i=0}^{\log_{\frac{2}{3}} n} n + T(1) * 3^{\text{Log}_{2/3} n}$$

$$T(n) = \text{Log}_{2/3} n * n + 1 * n^{\text{Log}_{2/3} 3}$$

$$T(n) = \text{Log}_{2/3} n * n + n^{\text{Log}_{2/3} 3}$$

عالآلة الحاسبة = -2.71

رح تكون قيمة ، بس انا رح اقارن الاكبر

$$T(n) = \text{Log}_{2/3} n * n + n^c$$

الـ T (n) هي رح اعتبرها الـ Running Time

باخذ اكبر power + بدون constant

Big O notation = O (n log n)

Example :

$$T(n) = 2T(n/2) + n^2$$

$a = 2$, $b = 2$, $T(n) = n^2$

الـ root هي $T(n) = n^2$ بتنقسم إلى 2 node

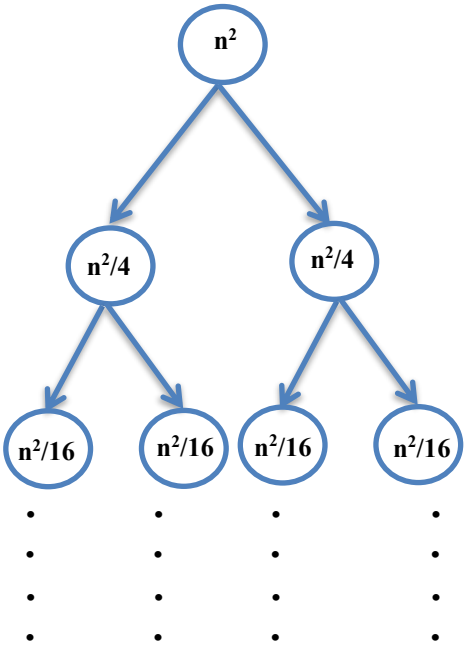
$$T(n) = 2T(n/2) + n^2$$

$$T(n/2) = 2T((n/2)/2) + (n/2)^2 =$$

$$= 2T(n/4) + n^2/4$$

$$T(n/4) = 2T((n/4)/4) + (n/4)^2$$

$$T(n/4) = 2T(n/16) + n^2/16$$



number of level $= \text{Log}_b\, n = \text{Log}_2\, n$

number of leaf on last level $= a^{\text{Log}_b\, n} = 2^{\text{Log}_2\, n} = n^{\text{Log}_2\, 2} = n$

$$T(n) = \sum_{i=0}^{h-1} n^2 + T(n) * n$$

$$T(n) = (n^2 + n^2/2 + n^2/4 + n^2/8 + \dots) + 1 * n$$

$$T(n) = n^2 \boxed{(1/2^0 + 1/2^1 + 1/2^2 + 1/2^3 + \dots)} + n$$

$$\sum_{i=0}^{h-1} \left(\frac{1}{2}\right)^i$$

$$T(n) = n^2 * \boxed{\sum_{i=0}^{h-1} \left(\frac{1}{2}\right)^i} + n$$

قد ما كان الناتج رح يكون الناتج عدد constant

$$1 / (1-r) = 1 / (1-1/2) = 1 / (1/2) = 2$$

$$T(n) = n^2 * 2 + n = 2n^2 + n$$

باخذ اكبر power + بدون constant

Big O notation = $O(n^2)$

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Binary Search

Binary Search : أشهر الطرق يلي بتستخدم طريقة الـ Divide & Conquer وهاي الطريقة انا بستخدمها بس بدي اقسم المشكلة لأجزاء مستقلة Big O اله O (log n)

(بقسم المشكلة لـ جزأين وبشتغل عالجزء المناسب) .

Linear Search : البحث الخطي بفحص عنصر عنصر بالمشكلة الموجودة عندي Big O اله O (n) .

Example :

الشرط الأساسي قبل ما استخدم الـ Binary Search لازم اتأكد من الـ array تكون مرتبة (sorted) إذا الـ array مش مرتبة ما بزبط استخدام Binary Search لأنه يعتمد على ترتيب العناصر .

1	2	10	12	30	73	90
low			mid			high

بحدد 3 مؤشرات أساسية في الـ array

low = مؤشر على بداية الـ array .

mid = مؤشر وسط الـ array .

$$mid = \frac{high - low}{2} + low$$

high = مؤشر على نهاية الـ array .

x = القيمة يلي بدي ادّور عليها بالـ array على فرض x = 2

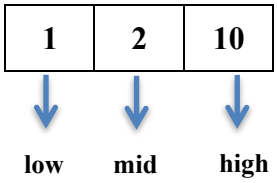
بفحص هل الـ x == mid ؟ 2 == 12 ؟ لا

طيب هل mid < x او mid > x ؟ 12 > 2 ، بلاقي قيمة x اصغر من الـ mid

إذا mid < x العنصر في جهة اليسار .

إذا mid > x العنصر في الجهة اليمين (تحقق الشرط اذاً بس بفحص بجهة اليمين) .

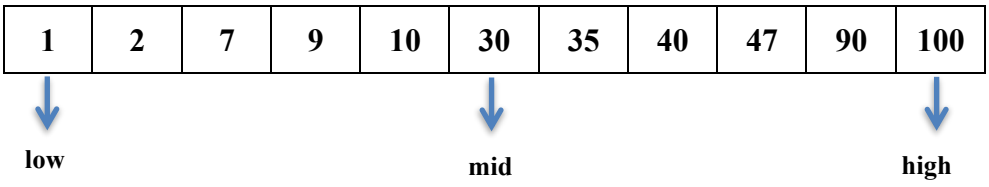
بس بفحص بجهة الـ array اليمين .



برجع بحدد الـ 3 مؤشرات عالـ array

بفحص هل الـ mid == x ؟ 2 == 2 ؟ اه وهيك لقيت العنصر .

Example :



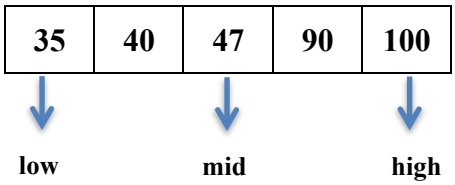
على فرض x = 90

بفحص هل الـ mid == x ؟ 30 == 90 ؟ لا

طيب هل mid < x او mid > x ؟ 90 > 30 ، بلاقي قيمة x اكبر من الـ mid

إذا mid < x العنصر في جهة اليسار (تحقق الشرط اذاً بس بفحص بجهة اليسار) .

إذا mid > x العنصر في الجهة اليمين .

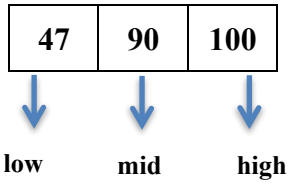


بفحص هل الـ mid == x ؟ 47 == 90 ؟ لا

طيب هل mid < x او mid > x ؟ 90 > 47 ، بلاقي قيمة x اكبر من الـ mid

إذا mid < x العنصر في جهة اليسار (تحقق الشرط اذاً بس بفحص بجهة اليسار) .

إذا mid > x العنصر في الجهة اليمين .



بفحص هل الـ mid == x ؟ 90 == 90 ؟ اه وهيك لقيت العنصر .

- **Sorted**

Binary search : O (Log n)

Linear search : O (n)

- **Unsorted**

Binary search + Big O of sorting : O (Log n) + O (n Log n)

عشان اخلي الـ array تكون sorted رح احتاج اسرع طريقة ترتيب مثلاً O (n Log n)

Linear search : O (n)

هاد الـ Big O بكل مرّة ببحث فيها عن عدد مرّة وحدة طلب واحد ، بس مثلاً لو كان عندي 5 طلبات بحث ؟

- **Unsorted**

Binary search : 5 * Log n + n Log n

Linear search : 5 * n

لو عندي m من الطلبات ؟

Binary search : m * Log n + n Log n

Linear search : m *n

على فرض $m = n^2$

Binary search : n^2 * Log n + n Log n = n^2 Log n + n Log n

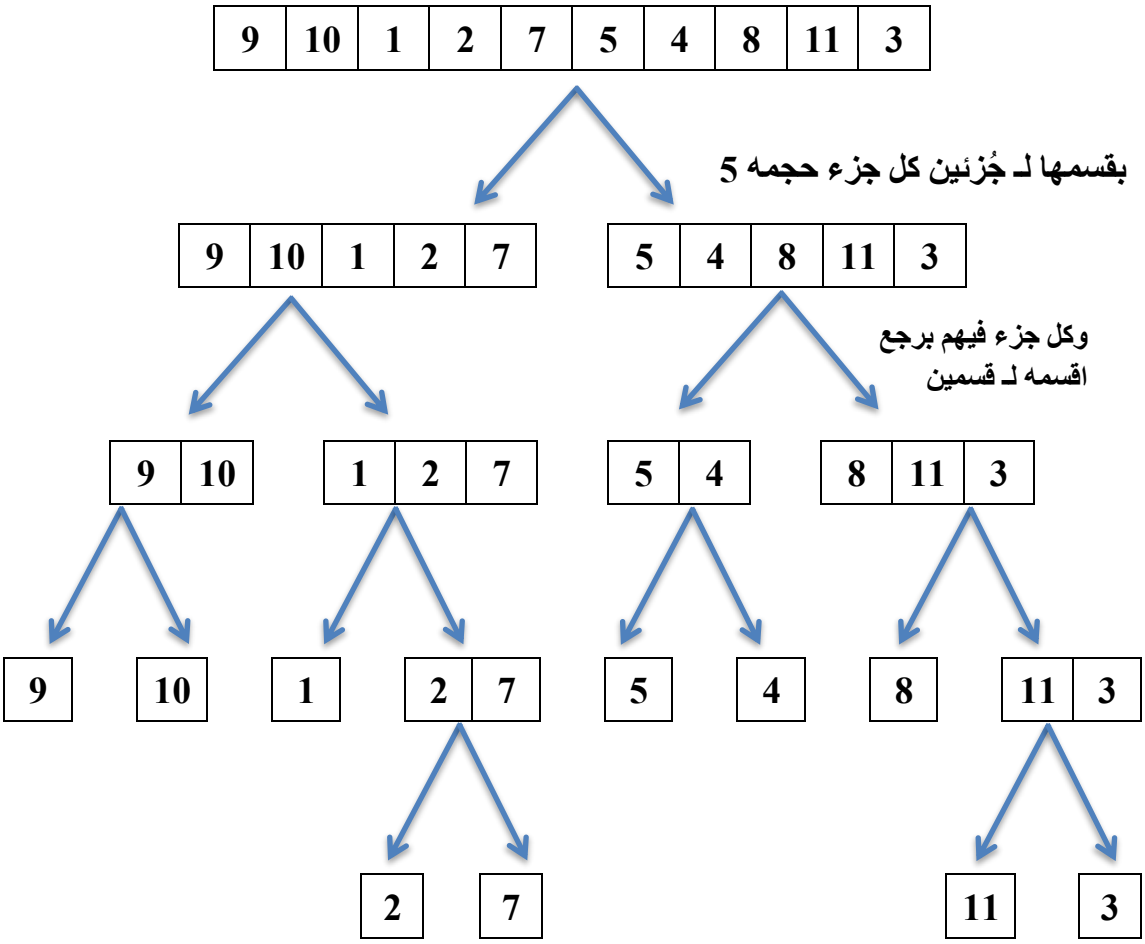
Linear search : n^2 * n = n^3

في حالات الطلبات الكبيرة الـ Binary search هو الأفضل .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Marge sort

بهاد النوع من الـ sort ، بكون عندي array بقسمها لَـ جُزئين مثلاً عندي array حجمها 10

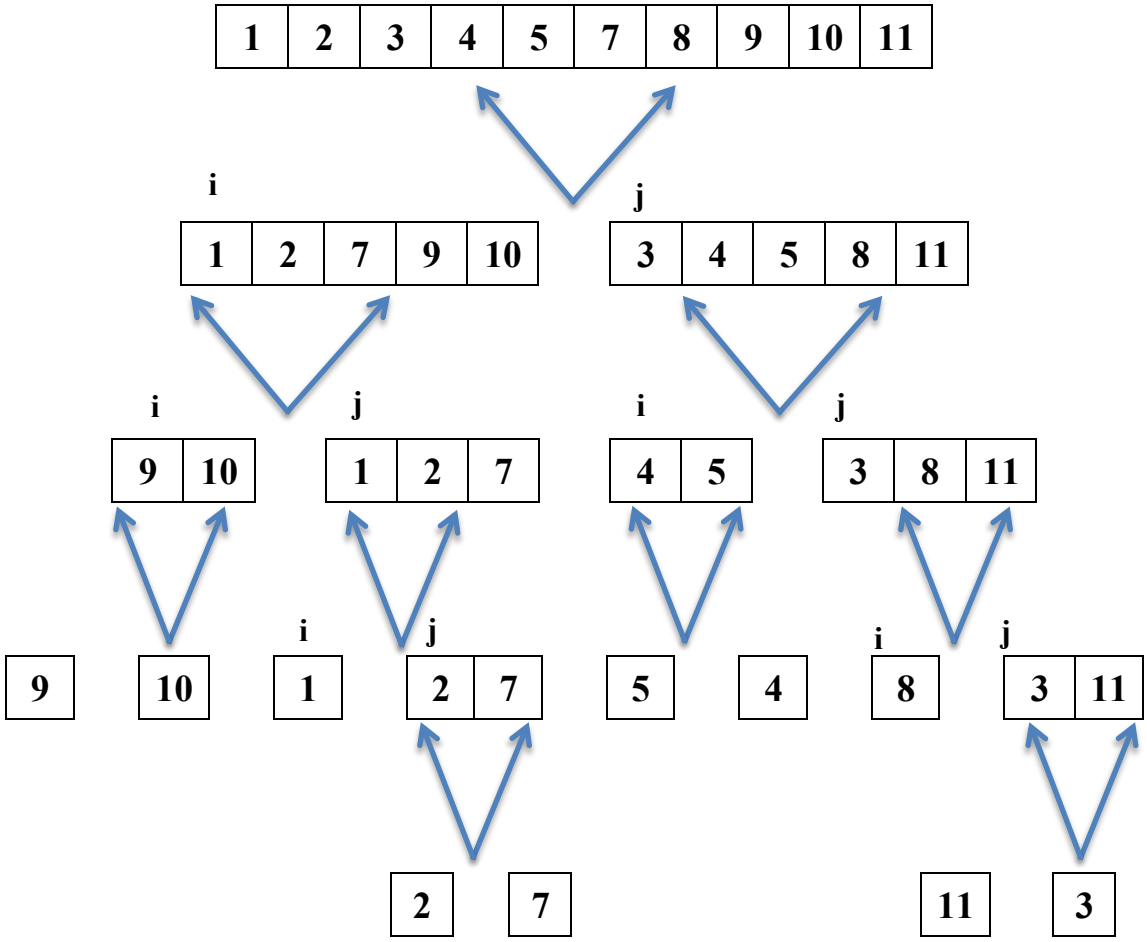


لـ أوصل لـ خانة .

انا هيك وصلت لـ عدة array كل وحدة منهم حجمها 1 وتُعتبر sorted .

بدي اعمل marge (دمج) ببلش هالمرة العكس من تحت لفوق وبقارن كل عنصر مين الأكبر بينتهم وبضيفه بالاول .

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

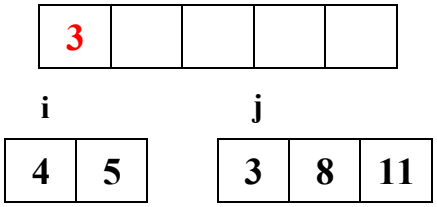


ببلش من تحت بقارن (بقارن بين خانة وخانة) مثلاً الـ 9 والـ 10 مين الأكبر ؟ بضيفه بالاول بس ادمج من جهة اليمين (بتكون الـ array بس خانتين بعد الدمج) .

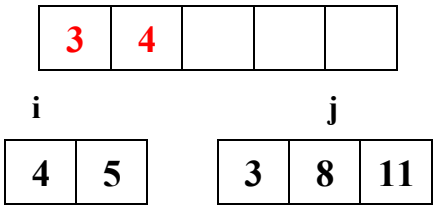
مثلاً المقارنة بين الـ 7 , 2 array والـ 1 array بعد الدمج رح أوصل لمقارنة بين خانة وبين array خانتين ، بحط مؤشر عند الـ index الأول من كل array بسميه i والثاني j وبقارن بين i و j الأكبر بينهم بضيفه هو اول .

مين الأكبر i = 1 او j = 2 ؟ 1 اكبر بضيفه اول عجهة اليمين بالـ array الجديدة يلي رح تكون من 3 خانات

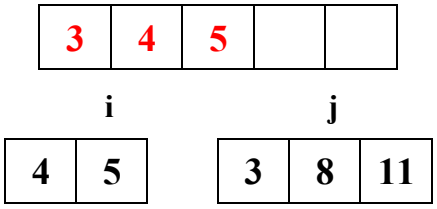
ولو في كمان عناصر بحرك المؤشر i او j عالـ index حسب لو عندي مثلاً array خانتين و 3 array خانات فلما قارنت i و j كانت i اصغر ضفتها بالـ array المدمجة رح احرك مؤشر i بس j ثابت وارجع اقارن بنفس الخطوات بحرك المؤشر بالـ array يلي رح اضيف منها .



بقارن بين 4 و 3 الأصغر 3 بضيفها وبحرك بس مؤشر الـ j بس .



بقارن بين 8 و 4 الأصغر صغر 4 بضيفها وبحرك بس مؤشر الـ i بس .



بقارن بين 8 و 5 الأصغر صغر 5 بضيفها وخلصت الـ array بضيف بعدها 8 و11 ولو كانت حجمها اكبر بضل امشي بنفس الخطوات .

بالتقسيم بتشبهه فكرة الـ log وهاي العملية بالتقسيم والتخزين تأخذ مساحة كبيرة ، وعملية الـ swap يلي بتصير عالـ array نفسها اسمها in place .

(المطلوب مني بالامتحان ارسم الـ array وتقسيمها قبل وبعد) .

عملية الدمج بكل level كانت تأخذ n

عدد الـ level هو Log n

Big O (n Log n)

Quick Sort

5	10	7	2	1	9

اول خطوة بختار Pivot (محور) ، عملية اختيار المحور هي أساس الحل .

الـ Pivot هو عنصر من عناصر الـ array احنا دايماً رح اختار الـ pivot يكون اول قيمة

(يلي قدامه اكبر منه و يلي وراه اصغر منه حتى لو مش مرتبات) .

Pivot = 5 بحط عأول قيمة بعد الـ Pivot مؤشر x وبحط كمان مؤشر عآخر قيمة y .

5	10	7	2	1	9
	X				Y

المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 10 اكبر من الـ 5 ؟ اه فـ بوقف

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 9 اقل من الـ 5 ؟ لا بكمل وبحرك المؤشر

5	10	7	2	1	9
	X			Y	

المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 10 اكبر من الـ 5 ؟ اه فـ بوقف

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 1 اقل من الـ 5 ؟ اه فـ بوقف

5	10	7	2	1	9
	X			Y	

هسا بهاي الخطوة x و y واقفين فـ بعمل Swap بين قيمة x و y (بس القيمة المؤشر ثابت) .

5	1	7	2	10	9
	X			Y	

المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) هل الـ 1 اكبر من الـ 5 ؟ لا بكمل وبحرك المؤشر

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) هل الـ 10 اقل من الـ 5 ؟ لا بكمل وبحرك المؤشر

وانا بحرك المؤشر بهاد الاتجاه بكون ادور
على عنصر اقل من الـ Pivot بوقف عنده

وانا بحرك المؤشر بهاد الاتجاه بكون ادور
على عنصر اكبر من الـ Pivot بوقف عنده

5	1	7	2	10	9
		X	Y		

المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 7 اكبر من الـ 5 ؟ اه فـ بوقف

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 2 اقل من الـ 5 ؟ اه فـ بوقف

هسا بهاي الخطوة x و y واقفين فـ بعمل Swap بين قيمة x و y (بس القيمة المؤشر ثابت) .

5	1	2	7	10	9
		X	Y		

المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 2 اكبر من الـ 5 ؟ لا بكمل وبحرك المؤشر

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 7 اقل من الـ 5 ؟ لا بكمل وبحرك المؤشر

5	1	2	7	10	9
Pivot		Y	X		

باللحظة يلي y بتتجاوز x بحركة المؤشر ببدل بين y و الـ Pivot .

2	1	5	7	10	9
Pivot		Y	X		

هيك الـ Pivot حقق المطلوب (يلي قبله اصغر منه ويلي بعده اكبر منه) .

برجع بكرر الخطوات بس هالمرّة بعد ما اقسم الـ array لقسمين .

7	10	9			
Pivot	X	Y			

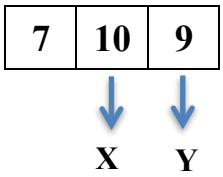
القسم يمين الـ Pivot

5	
جاهز	

2	1
Pivot	X / Y

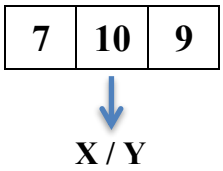
القسم يسار الـ Pivot

القسم يمين الـ Pivot الـ Pivot = 7



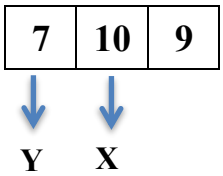
المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 10 اكبر من الـ 7 ؟ اه فـ بوقف

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 9 اقل من الـ 7 ؟ لا بكمل وبحرك المؤشر



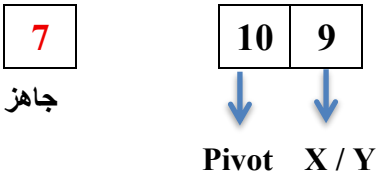
المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 10 اكبر من الـ 7 ؟ اه فـ بوقف

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 9 اقل من الـ 7 ؟ لا بكمل وبحرك المؤشر

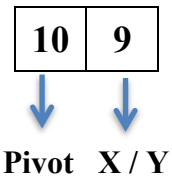


باللحظة يلي y بتتجاوز x بحركة المؤشر ببدل بين y و الـ Pivot (نفس المكان رح يضلو) .

برجع بكرر الخطوات بس هالمرّة بعد ما اقسم الـ array لقسمين .



هيك الـ Pivot حقق المطلوب (يلي قبله اصغر منه ويلي بعده اكبر منه) .



بنفس الطريقة رح ارجع اكرر الخطوات

المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 9 اكبر من الـ 10 ؟ لا بكمل وبحرك المؤشر

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 9 اقل من الـ 10 ؟ اه فـ بوقف

10	9

Pivot Y X

باللحظة يلي y بتتجاوز x بحركة المؤشر ببدل بين y و الـ Pivot

9	10

Pivot Y

هيك الـ Pivot حقق المطلوب (يلي قبله اصغر منه ويلي بعده اكبر منه) .

بدمج الـ array بعد الترتيب .

5	7	10	9

- القسم يسار الـ Pivot الـ Pivot = 2

2	1

Pivot X / Y

المؤشر x (بدور على عدد اكبر من الـ Pivot أوقف عنده) الـ 1 اكبر من الـ 2 ؟ لا بكمل وبحرك المؤشر

المؤشر y (بدور على عدد اقل من الـ Pivot أوقف عنده) الـ 1 اقل من الـ 2 ؟ اه فـ بوقف

2	1

Pivot Y X

باللحظة يلي y بتتجاوز x بحركة المؤشر ببدل بين y و الـ Pivot

1	2

Pivot Y

برجع ادمج المصفوفة الأولى بعد الترتيب

1	2	5	7	9	10

Sorted

	Marge Sort	Quick Sort
Big O notation (worst case)	**n log n**	**n^2**
Big Ω notation (best case)	**n log n**	**n log n**
Big Θ notation (average case)	**n log n**	**n log n**

Type	Worst Case O	Best Case Ω	Average Case
Linear search	$O(n)$	1	
Selection sort	$O(n^2)$	n^2	n^2
Bubble sort	$O(n^2)$	n	
Insertion sort	$O(n^2)$	n	n^2
Heap sort	$O(n\log n)$.	$n\log n$	$n\log n$
Merege sort	$O(n\log^2 n)$	$n\log^2 n$	$n\log^2 n$.
Quick sort	$O(n^2)$	$n\log n$	$n\log n$
Binary search →	sorted $\log n$	$O(1)$	$\log n$.
→	unsorted $\log n + m\log n$		
Graph → DFS	$O(V+E)$		
→ BFS			

"اسْتَغْفِرُوا رَبَّكُمْ إِنَّهُ كَانَ غَفَّارًا"

Graph

Graph : هو بُنية بيانات / او نموذج رياضي بمثل كائنات وعلاقاتها بكتبه عادةً كـ G (V , E)

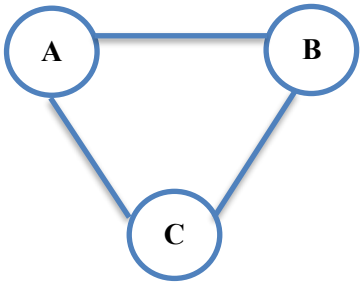
V : مجموعة العُقد الـ nodes او القمم الـ vertices .

E : مجموعة الحواف الـ edges يلي بتربط العُقد الـ nodes ببعض .

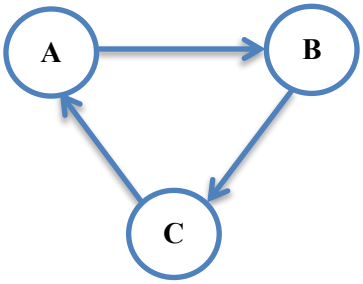
- الأنواع من حيث الاتجاه .

موجّهة (Directed) : العلاقة بين الـ node محددة باتجاه معين اتجاه واحد او اكثر حسب اتجاه الاسم بين الـ node .

غير موجّهة (Undirected) : العلاقة بين الـ node مش محددة باتجاه معين اتجاه مزدوج .



Undirected : مثلاً الاتجاه مش محدد بالمسارات عادي ممكن امشي بالاتجاهين من A لَـ B والعكس .



Directed : مثلاً المسار من A لَـ B بس بمشي بمساره مسار واحد باتجاه محدد حسب السهم .

- الأنواع من حيث الاوزان .

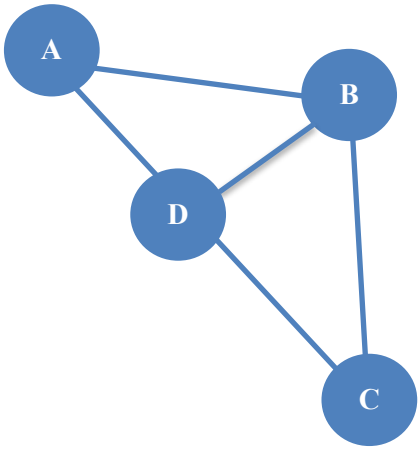
Weight : قيمة بتكون عدد مكتوب فوق الـ edge وهيّ تكلفة الانتقال بين كل node والثانية .

Un Weight: ما بكون مكتوب وما في قيمة .

لتمثيل الـ Graph جوا الـ Computer في طريقتين

Adjacency Matrix : بالتمثيل بمثلهم كأنهم مصفوفة مرتبة ، اذا في edge بين كل node والثانية بحط 1 اذا ما في بحط 0 وبدي انتبه للاتجاه .

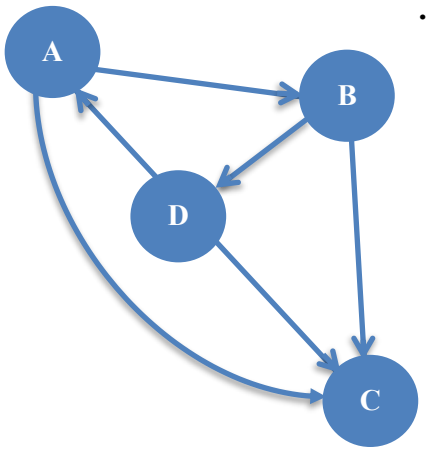
لو كان Undirect



	A	B	C	D
A	0	1	0	1
B	1	0	1	1
C	0	1	0	1
D	1	1	1	0

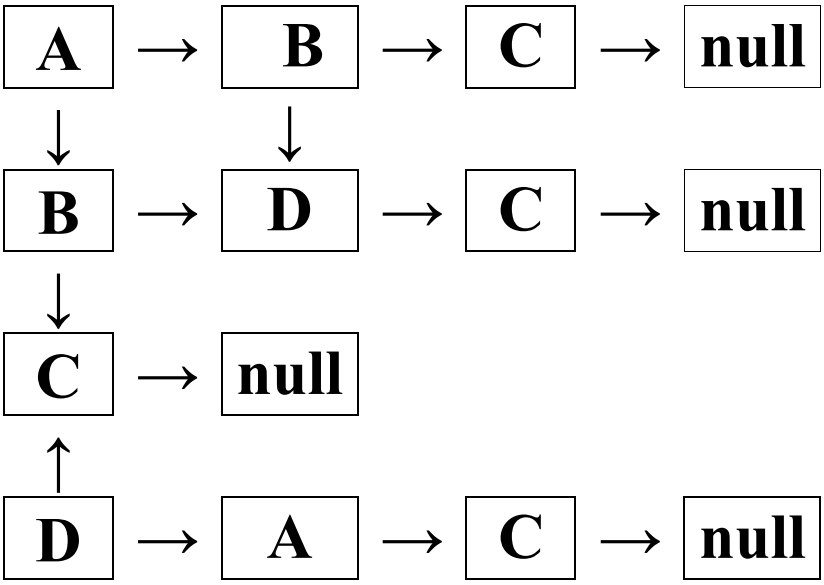
لو كان weight Graph بكتب القيمة يلي على edge مكان الـ 1 .

لو كان Direct Graph بنتبه اذا في Edge واتجاه السهم بكل node .

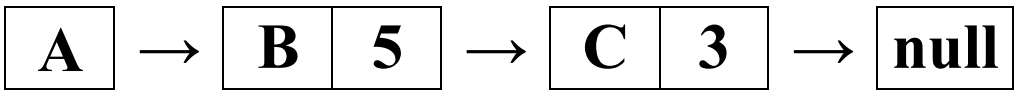


	A	B	C	D
A	0	1	1	0
B	0	0	1	1
C	0	0	0	0
D	1	0	1	0

Adjacency List : على نَفس الشكل Direct Graph كل node بتحافظ ع قيمة الـ node الموصولة معها وبتضل ماسكة فيها .



لو كان weight Graph بكتب القيمة يلي على edge بهاي الطريقة .

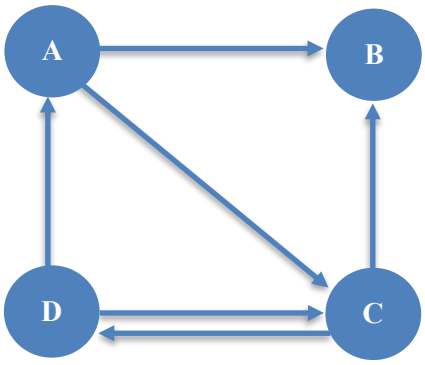


المسافة بين الـ A و B = 5

المسافة بين الـ B و C = 3

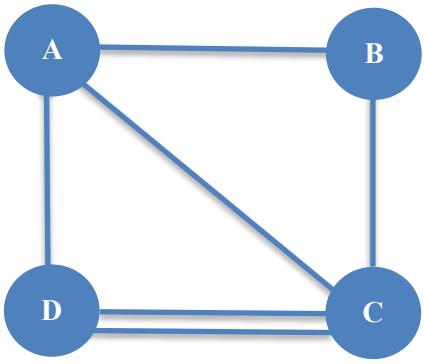
Example :

الرسمة Direct



	A	B	C	D
A	0	1	1	0
B	0	0	0	0
C	0	1	0	1
D	1	0	1	0

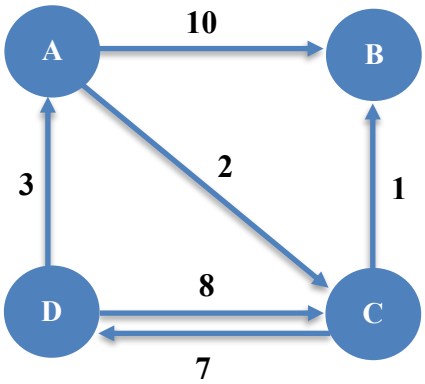
الرسمة Undirect



	A	B	C	D
A	0	1	1	1
B	1	0	1	0
C	1	1	0	2
D	1	0	2	0

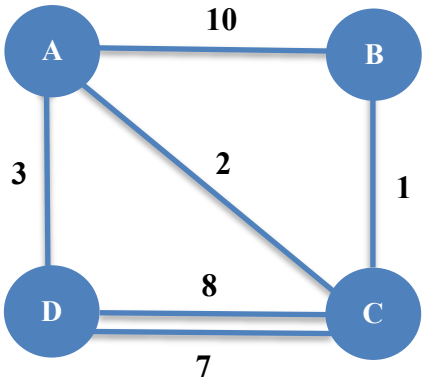
1 + 1 = 2 لانه في طريقين من C لَـ D .

الرسمة Direct وكان عنا wight



	A	B	C	D
A	0	10	2	0
B	0	0	0	0
C	0	1	0	8
D	3	0	7	0

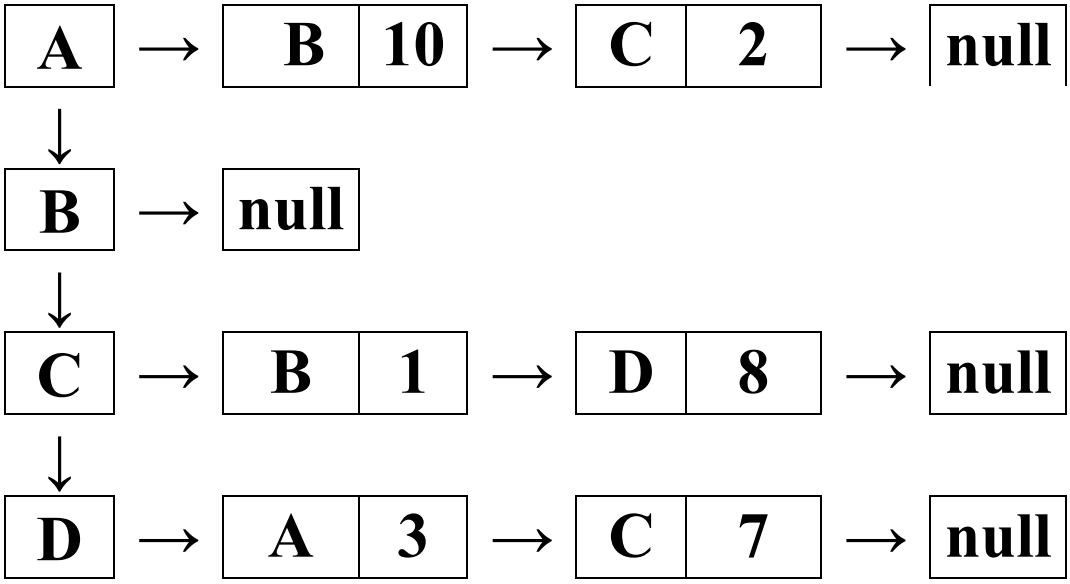
الرسمة Undirect وكان عنا wight



	A	B	C	D
A	0	10	2	3
B	10	0	1	0
C	2	1	0	15
D	3	0	15	0

8 + 7 = 15 لانه في طريقين من C لَـ D .

تمثيل weight Graph بطريقة الـ Adjacency List



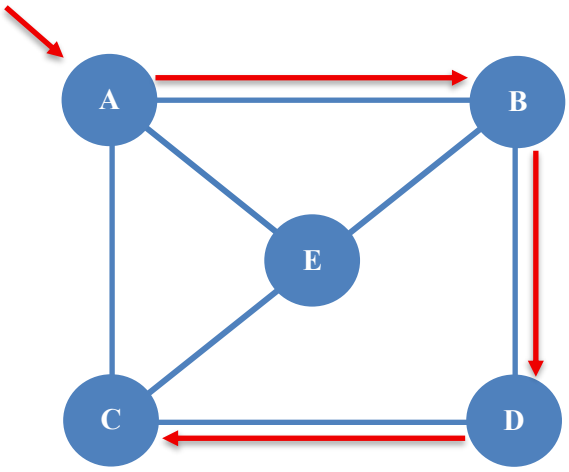
DFS

DFS (Depth-First Search) – البحث بالعمق

يعتمد على مبدأ الـ Stack

بمشي في خط واحد لأبعد نقطة بعدين يرجع لـ ورا اكمل باقي التفرعات مناسب لما بدي اوصل لـ نقطة بعيدة أو حل بتطلب التفرع العميق مثل البحث في متاهة لـ الوصول إلى نهاية بعيدة .

نقطة البداية .



ببلش من A باعتبارها نقطة الانطلاق في الرسم البياني (او الدكتور بحدد نقطة البداية) هاي أول node بزورها وببلش منها الاستكشاف هسا عندي 3 مسارات طالعة من A

من A لـ B

ومن A لـ C

ومن A لـ E

الـ A متصلة بـ 3 node (B و C و E بسميهم جيران الـ node A) بالـ DFS بختار واحد من هدول الجيران بمشي فيه حتى النهاية بعدها برجع بمشي لباقي الجيران بختار المسار حسب ترتيب الحروف هاي اسهل طريقة ولو كان في Weight باخذ مسار اقل عدد .

بستخدم مصفوفة زيارات Visited Array ، بالبداية بتكون كلها أصفار وكل ما أزور node بحط 1 ، وكل ما أزور node بطبعها .

A	B	C	D	E
0	0	0	0	0

A	B	C	D	E
1	1	0	0	0

AB

هسا عندي B كم مسار طالع منها ؟

من B لـ A

من B لـ D

ومن B لـ E

من المسارات مين فيهم زرتها ؟ A ، ما برجع امشي بهاد المسار بضل عندي اما من B لـ D او من B لـ E بختار حسب الترتيب الابجدي (D , E) ؟ D بالاول بزورها وبحط مكانها بالمصفوفة 1 وبطبعها .

A	B	C	D	E
1	1	0	1	0

A B D

هسا عندي D كم مسار طالع منها ؟

من D لـ C

من D لـ B

من المسارات مين فيهم زرتها ؟ B ، ما برجع امشي بهاد المسار بضل عندي من D لـ C بزورها وبحط مكانها بالمصفوفة 1 وبطبعها .

A	B	C	D	E
1	1	1	1	0

A B D C

هسا عندي C كم مسار طالع منها ؟

من C لـ E

من C لـ D

ومن C لـ A

من المسارات مين فيهم زرتها ؟ D و A ، ما برجع امشي بهدول المسارات بضل عندي من C لـ E بزورها وبحط مكانها بالمصفوفة 1 وبطبعها .

A	B	C	D	E
1	1	1	1	1

A B D C E

بس المصفوفة تكون كلها 1 يعني زرت الكل وهاد الترتيب A B D C E .

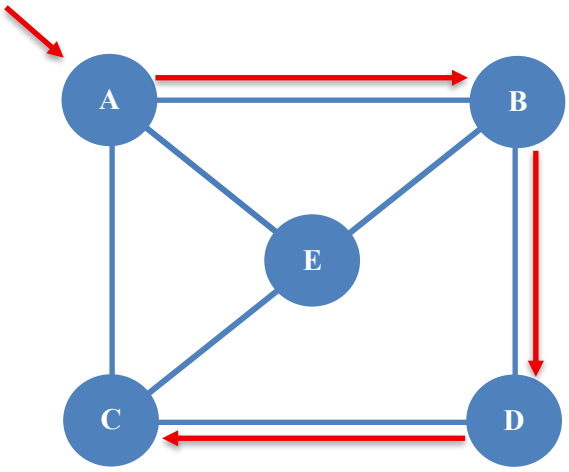
BFS

البحث بالعرض – BFS (Breadth-First Search)

بعتمد على مبدأ الـ Queue

بستكشف كل الجيران أولاً قبل ما اروح لـ المستوى الثاني ، مناسب لما بدي أقصر مسار أو حل بتطلب التدرج مثل إيجاد أقصر طريق بين نقطتين في شبكة طرق .

نقطة البداية .



ببلش من A باعتبارها نقطة الانطلاق في الرسم البياني (او الدكتور بحدد نقطة البداية) هاي أول node بزورها وببلش منها الاستكشاف هسا عندي 3 مسارات طالعة من A

من A لـ B

ومن A لـ C

ومن A لـ E

الـ A متصلة بـ 3 node (B و C و E بسميهم جيران الـ node A) بالـ BFS بمشي مرّة وحدة من كل هدول الجيران يعني بزورهم مرّة وحدة .

يعني بزور B و C و E وبحط مكانهم 1 بالمصفوفة وبطبعهم .

A	B	C	D	E
1	1	1	0	1

A B C E

بعدها باخذ الحرف الابجدي بعد الـ A يعني B كم مسار طالع منها ؟

من B لـ A

من B لـ D

ومن B لـ E

من المسارات مين فيهم زرتها ؟ A و E بضل D بزورها وبحط مكانها بالمصفوفة 1 وبطبعها .

A	B	C	D	E
1	1	1	1	1

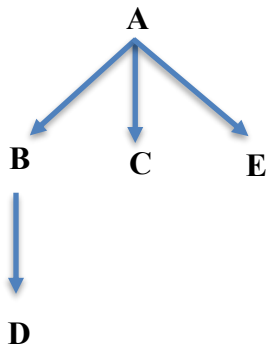
A B C E D

لو بدي اكمل رح اكمل من مسار B يعني كان عندي تفرع لـ E و لـ D

بالمسار E بتوديني ع C بس زرتها (يعني بفحص المسار كامل) .

وبرضو بالمسار D بتوديني ع C بس زرتها .

او شكل الشجرة



A B C E D

" يَرْفَعِ اللهُ الذينَ آمَنُوا مِنكُمْ وَالذينَ أُوتُوا العِلمَ دَرَجَاتٍ "

في الختام ما من فضلٍ إلا كان بتوفيقٍ من الله وحده ، وما علمٍ نِلتُه إلا بِرحْمةٍ ساقها الله إليّ ، فلهُ الحَمدُ كما ينبغي لجلالهِ وعظيم سلطانه فالحمدلله على عظيم نِعمه .